

Report: Optimising NYC Taxi Operations

Include your visualisations, analysis, results, insights, and outcomes. Explain your methodology and approach to the tasks. Add your conclusions to the sections.

1. Data Preparation

1.1. Loading the dataset

1.1.1. Sample the data and combine the files

The twelve monthly 2023 Yellow Taxi Parquet files were processed sequentially to limit memory use. After converting pickup timestamps to datetime, records with invalid or missing pickup times were removed. Trips were sampled within each pickup-date and pickup-hour group at a 0.72% rate using a fixed random seed of 42. This stratified approach was intended to retain representation across different days and hours while reducing the dataset to a manageable size. Monthly samples were combined, helper columns were removed, and the resulting annual sample was shuffled and saved in Parquet format for subsequent analysis.

2. Data Cleaning

2.1. Fixing Columns

2.1.1. Fix the index

Column names were standardized by removing surrounding whitespace, converting names to lowercase, and replacing spaces with underscores. Selected fields were then renamed to consistent snake_case names, such as `vendor_id`, `rate_code_id`, `pu_location_id`, and `do_location_id`. The DataFrame index was reset with `drop=True` so the old index is not retained as an extra column.

2.1.2. Combine the two airport_fee columns

The dataset contained two airport-fee columns with inconsistent capitalization. Because both appear to represent the same TLC airport-fee field, they were consolidated into a single `airport_fee_combined` column, and the original columns were removed. Missing values were treated as no recorded fee.

2.2. Handling Missing Values

2.2.1. Find the proportion of missing values in each column

The proportion of missing values was calculated for each column as the number of null entries divided by the total number of rows, expressed as a percentage. The initial missing-value summary was generated before imputation, so it records the original missingness. Use the displayed summary table for the column-specific percentages; no figures are stated here because the notebook output was not provided.

2.2.2. Handling missing values in passenger_count

Missing passenger_count values were imputed using the median of valid passenger counts from 1 to 6. Values of zero or less were also replaced with this median because they are not valid passenger counts.

2.2.3. Handle missing values in RatecodeID

RatecodeID is a categorical field identifying the fare rate applied to a trip. Missing values were replaced with 1, representing the Standard rate. This preserves the affected records and assigns them a valid rate category. The value counts are displayed after imputation to verify the resulting distribution. This replacement is an assumption: trips with missing codes may not necessarily have used the Standard rate, so rate-code comparisons should be interpreted with that limitation in mind..

2.2.4. Impute NaN in congestion_surcharge

Missing values in congestion_surcharge were replaced with 0, assuming that a missing surcharge indicates it was not applied. The code then checks the column's remaining missing-value count to verify the imputation.

2.3. Handling Outliers and Standardising Values

2.3.1. Check outliers in payment type, trip distance and tip amount columns

The outlier checks flag records with a payment_type of 0, which is not a valid category in the TLC data dictionary. They also flag trips with a trip_distance greater than 250 miles, as this is considered implausible for a New York City taxi trip. Records with both zero distance and zero fare but different pickup and drop-off zones are flagged as inconsistent.

The code removes rows meeting any of these conditions, then separately removes records with more than six passengers. The flagged-row counts are printed during execution; those results should be used to report how many records were excluded.

Although negative monetary values, including negative `tip_amount` values, are addressed in the earlier cleaning steps, this outlier code does not check for unusually high tip amounts. Therefore, no conclusion about extreme positive tips can be drawn from these checks.

3. Exploratory Data Analysis

3.1. General EDA: Finding Patterns and Trends

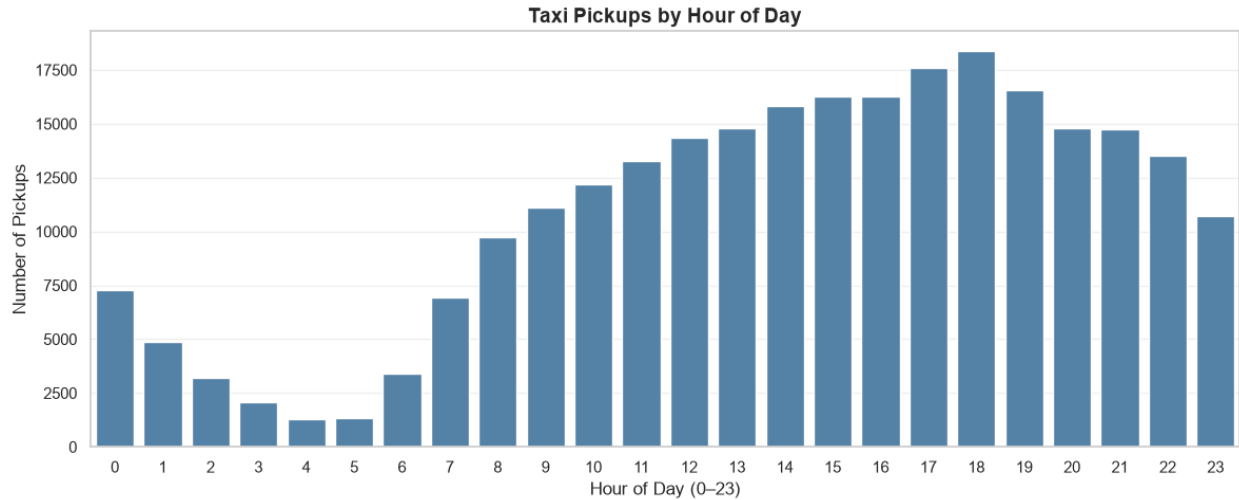
3.1.1. Classify variables into categorical and numerical (Pasted screenshot from Jupiter Notebook)

Variable	Category	Reasoning
<code>vendor_id</code>	Categorical (Nominal)	Codes representing which TPEP provider (e.g., 1, 2) — labels, not quantities
<code>tpep_pickup_datetime</code>	Numerical (Continuous) — often treated as Datetime/Temporal	Represents a point in time; used to derive numerical/categorical features (hour, day, etc.)
<code>tpep_dropoff_datetime</code>	Numerical (Continuous) — often treated as Datetime/Temporal	Same as above
<code>passenger_count</code>	Numerical (Discrete)	Whole-number counts of people
<code>trip_distance</code>	Numerical (Continuous)	Measured in miles, can take any decimal value
<code>rate_code_id</code>	Categorical (Nominal)	Represents fare-type codes (Standard, JFK, Newark, etc.), not a quantity
<code>pu_location_id</code>	Categorical (Nominal)	Zone ID label, not a measurable quantity
<code>drop_location_id</code>	Categorical (Nominal)	Same as above
<code>payment_type</code>	Categorical (Nominal)	Represents payment method labels (Credit card, Cash, etc.)
<code>pickup_hour</code>	Categorical (Ordinal) — sometimes treated as Numerical (Discrete)	Represents hour of day (0–23); has a natural order but is often used as a grouping variable rather than a quantity to sum/average meaningfully
<code>trip_duration</code>	Numerical (Continuous)	Measured in minutes/seconds, computed from timestamps

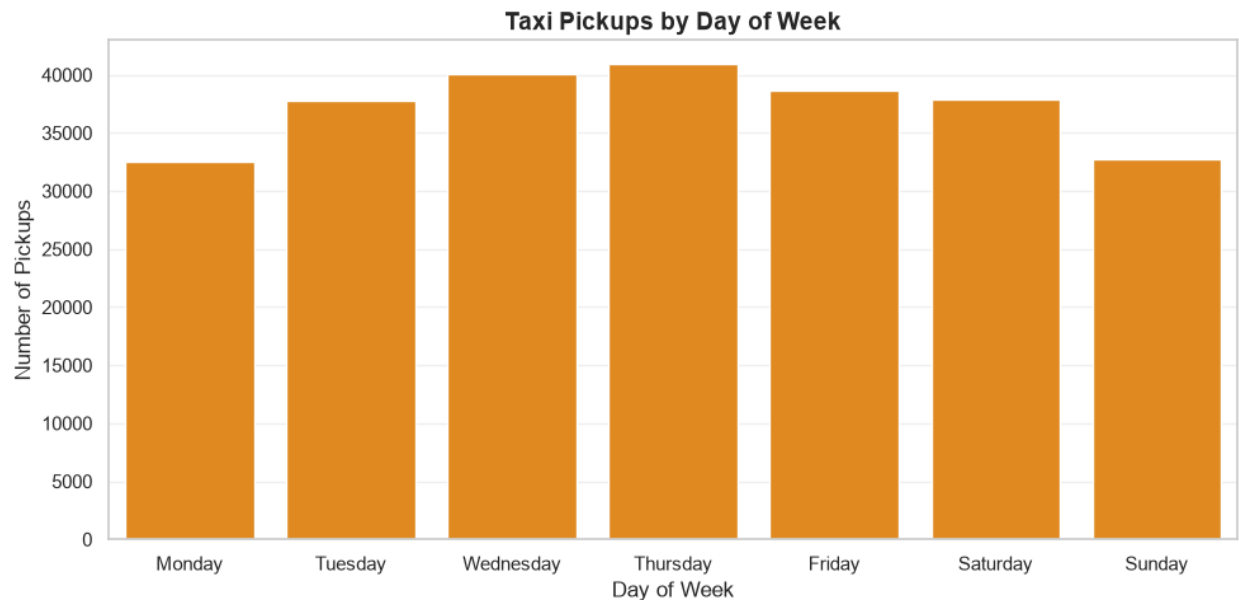
3.1.2. Analyze the distribution of taxi pickups by hours, days of the week, and months

The analysis used the cleaned, sampled taxi dataset. Pickup records were counted separately by hour, day of the week, and month. The day and month categories were ordered chronologically so their plots would be easier to interpret.

Hourly pickups: The chart shows a clear overnight low, with the fewest pickups around 4–5 a.m. Counts rise through the morning and afternoon, reaching a peak at 6 p.m. (about 18,000 pickups), then decline through the evening. This pattern suggests demand is concentrated later in the day, with a notable evening rush.



Pickups by day of the week: Thursday has the highest count (about 41,000), followed by Wednesday. Monday and Sunday have the lowest totals, at roughly 32,000–33,000. Since this chart compares total counts, it does not adjust for how many of each weekday occur in the dataset.



Monthly pickups: May has the highest count (about 24,000), with October also high. Counts are lowest in August and September, at approximately 19,000, before rising in October. Overall, the chart suggests some month-to-month variation rather than a steady increase or decrease across the year.



3.1.3. Filter out the zero/negative values in fares, distance and tips

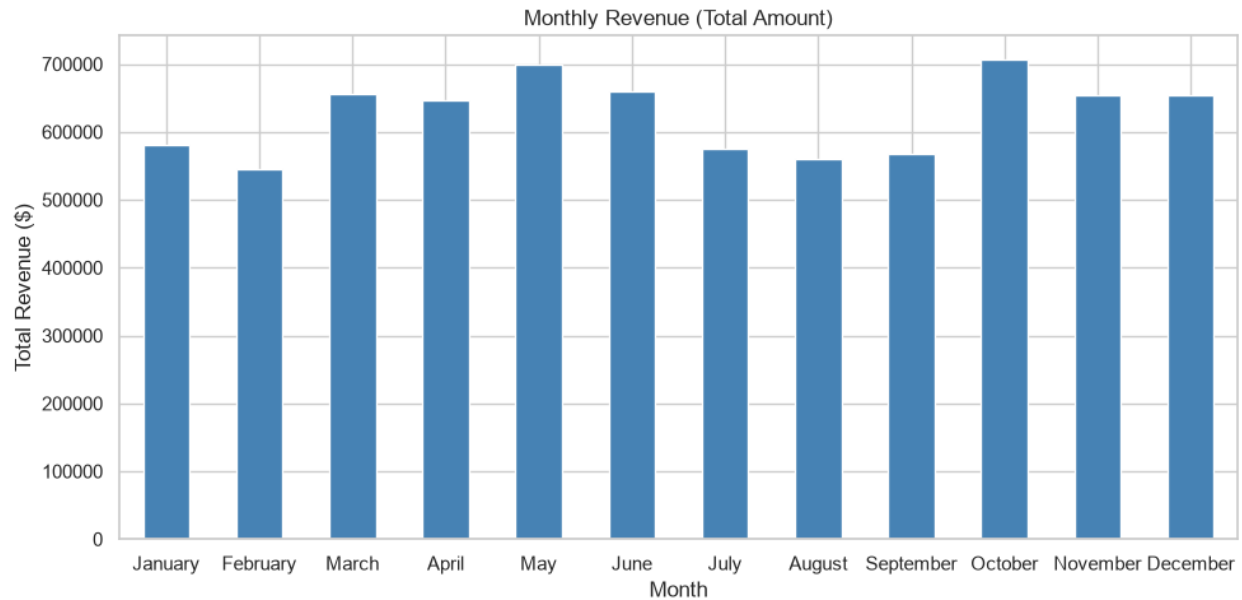
A separate `financial_df` was created for financial analysis, leaving `sampled_df_clean` unchanged for other analyses. Trips with zero or negative `fare_amount`, `total_amount`, or `trip_distance` were excluded because they do not represent a valid positive fare or distance and can distort distance-based financial measures. Negative monetary values had already been removed during data cleaning.

Zero tips were retained: a passenger may legitimately choose not to tip. The code therefore filters for `tip_amount >= 0`, excluding negative tips while preserving zero-tip trips. The resulting dataset contains trips suitable for financial and distance-based analysis.

3.1.4. Analyse the monthly revenue trends

Monthly revenue was analysed using `financial_df`, the dataset prepared for financial analysis by excluding trips with non-positive fare, total amount, or trip distance. The records were grouped by pickup month, and `total_amount` was summed for each month. The monthly totals were then plotted as a bar chart to compare revenue across the year.

`total_amount` was used as a measure of trip revenue because it captures the amount charged to passengers, including applicable fares and charges. The analysis indicates how revenue varies month to month within the sampled dataset. Since the calculation uses sampled records and does not scale the totals to the full dataset, the chart should be interpreted as a comparison of sampled monthly revenue—not actual citywide revenue.

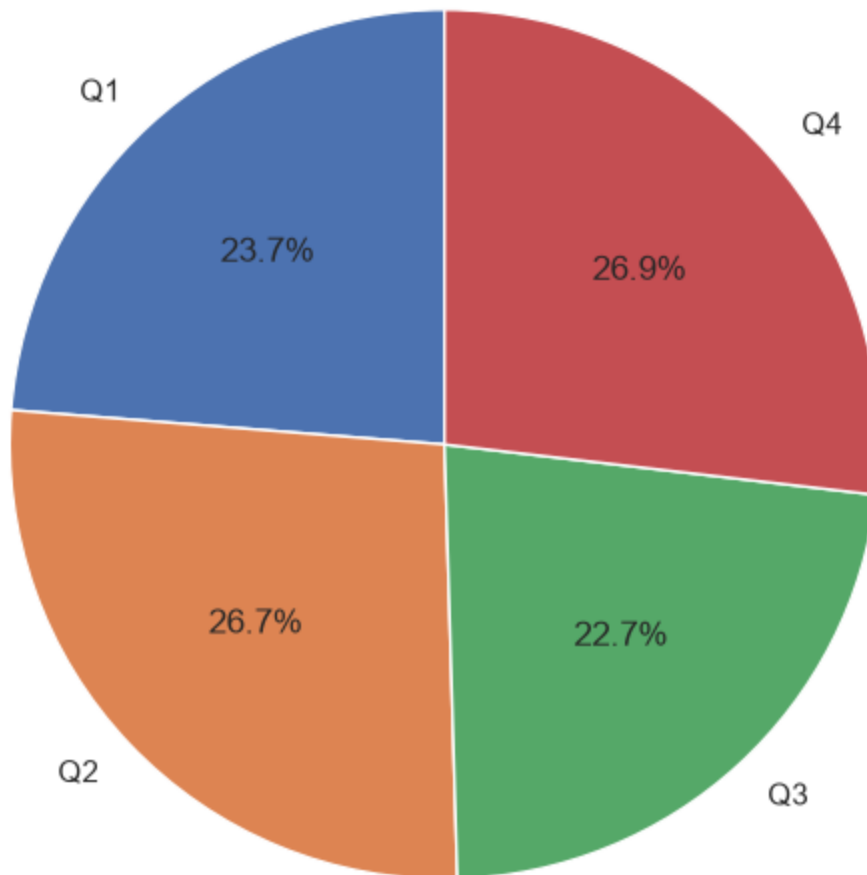


3.1.5. Find the proportion of each quarter's revenue in the yearly revenue

To compare each quarter's contribution to annual revenue, the pickup timestamp was used to assign every trip in `financial_df` to a calendar quarter (Q1–Q4). The `total_amount` values were then summed within each quarter. Each quarterly total was divided by the sum of revenue across all four quarters and multiplied by 100 to calculate its percentage share. The percentages were rounded to two decimal places and displayed in a pie chart.

The analysis uses the filtered financial dataset: trips with positive `fare_amount`, `total_amount`, and `trip_distance` were included, while zero-tip trips were retained. Since the dataset is a sample, the chart shows each quarter's share of revenue in the analysed sample, not an unscaled estimate of total citywide revenue. The `total_amount` field also does not include cash tips.

Proportion of Revenue by Quarter

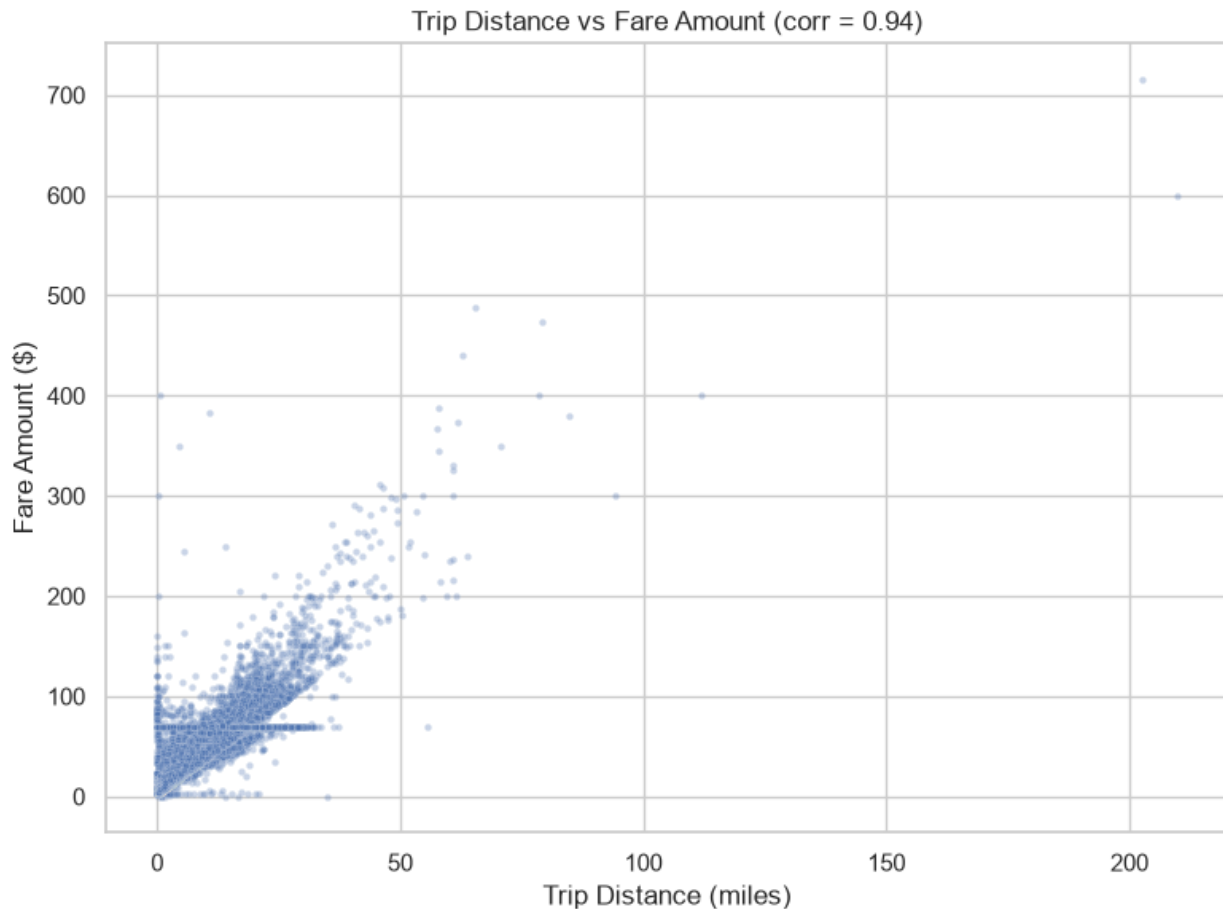


3.1.6. Analyse and visualise the relationship between distance and fare amount

To examine how trip distance relates to fare, the analysis created `dist_fare_df` from `financial_df`, retaining trips with a positive `trip_distance`. It then calculated the Pearson correlation between `trip_distance` and `fare_amount` using Pandas' `.corr()` method. The coefficient was printed to four decimal places.

A scatter plot was used to visualise the relationship, with trip distance in miles on the x-axis and fare amount in dollars on the y-axis. The points were plotted with low opacity (`alpha=0.3`) to make overlapping observations easier to see. The correlation was also included in the plot title, rounded to two decimal places.

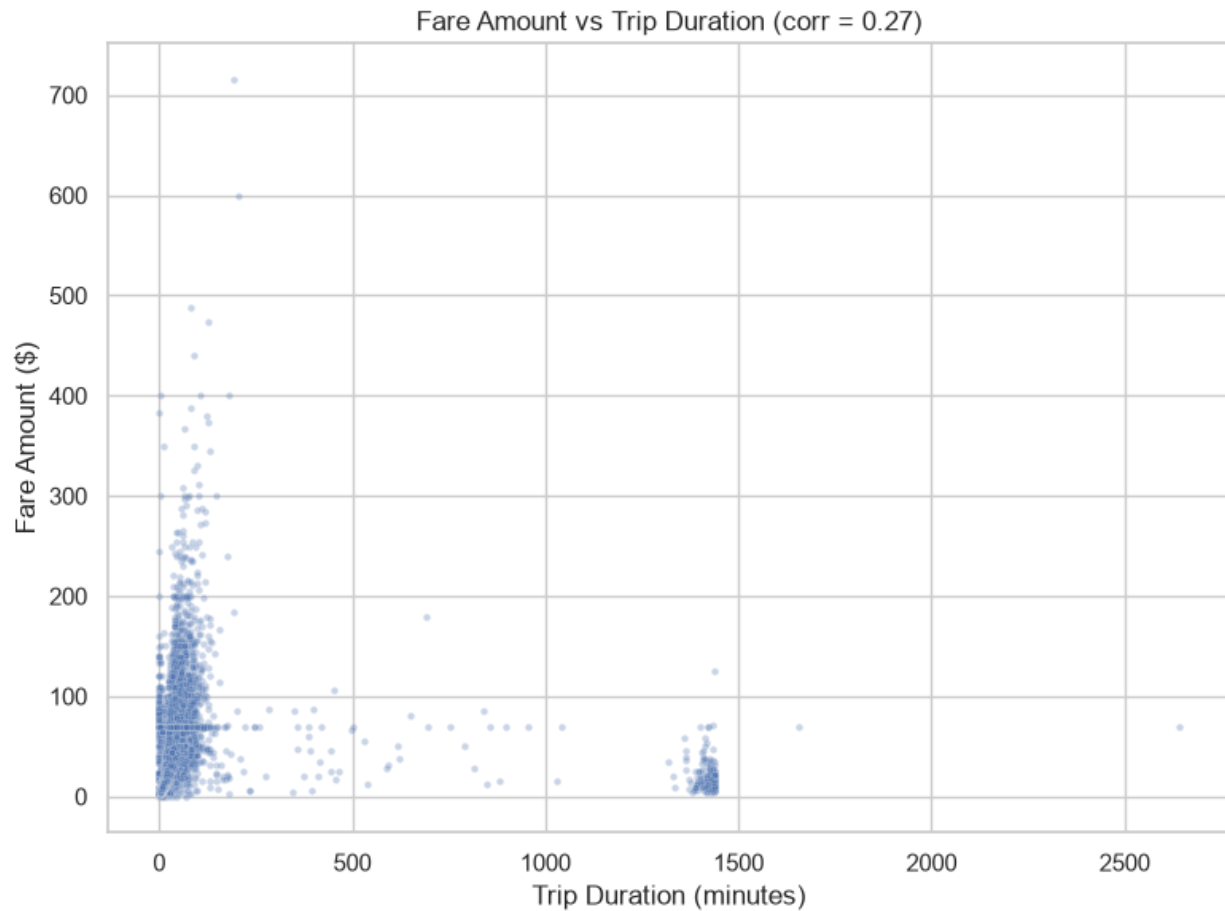
The plot and correlation indicate whether fares tend to increase as trips become longer. The correlation value itself should be reported from the cell's output; it is not available in the provided code excerpt. Correlation describes association, not causation, and the spread of points may also reflect factors such as traffic, surcharges, and fare rules.



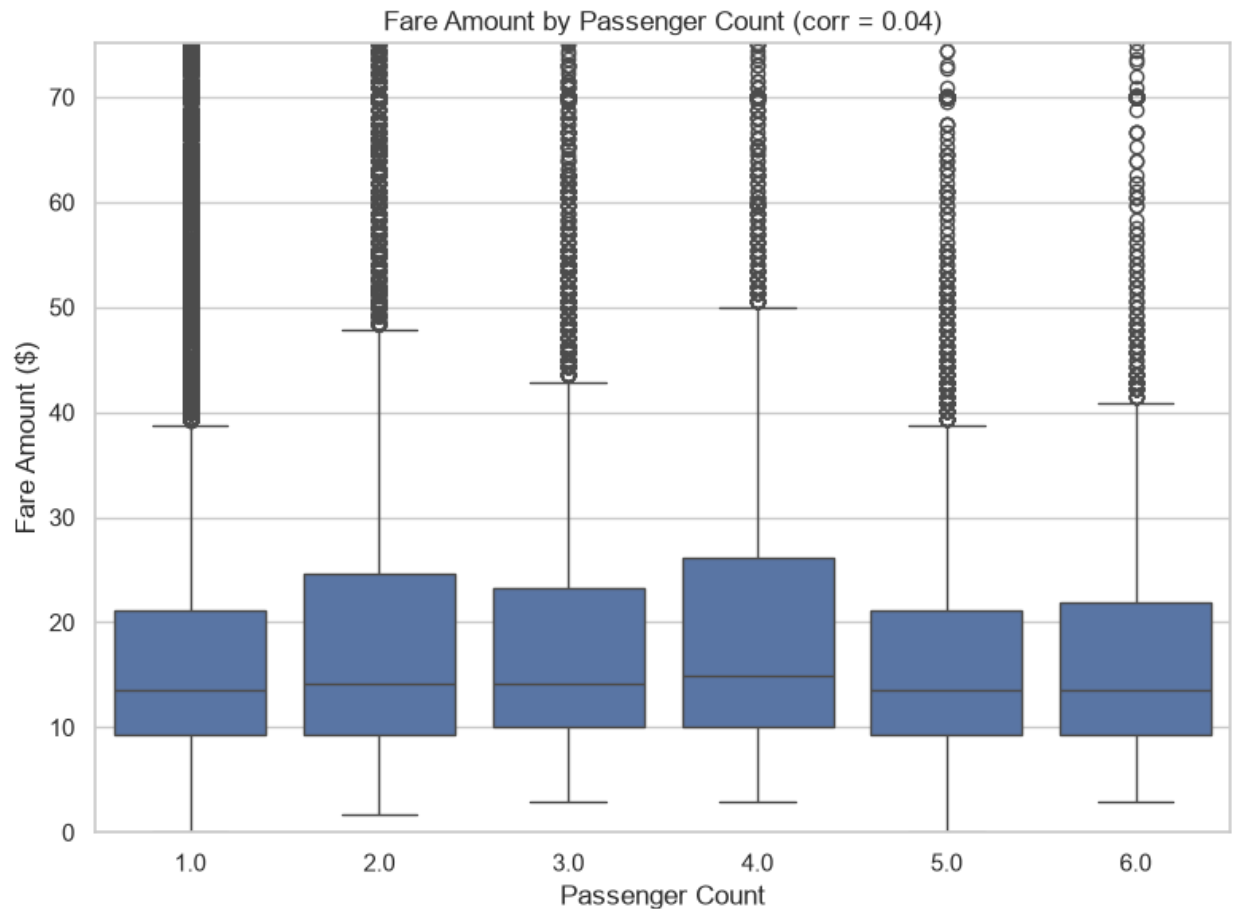
3.1.7. Analyse the relationship between fare/tips and trips/passengers

The analysis used `financial_df`, which contains trips retained for financial analysis. For each pair of variables, Pearson's correlation coefficient was calculated using Pandas' `.corr()` method. This coefficient measures the direction and strength of a linear association; it does not establish causation.

Fare amount and trip duration: Trip duration was calculated in minutes by subtracting the pickup time from the drop-off time. The correlation between duration and fare_amount was then calculated, and a scatter plot was created with duration on the x-axis and fare on the y-axis. The points were plotted with transparency to make overlapping observations easier to see. This visualisation helps assess whether longer trips tend to have higher fares and how much variation exists at similar durations.



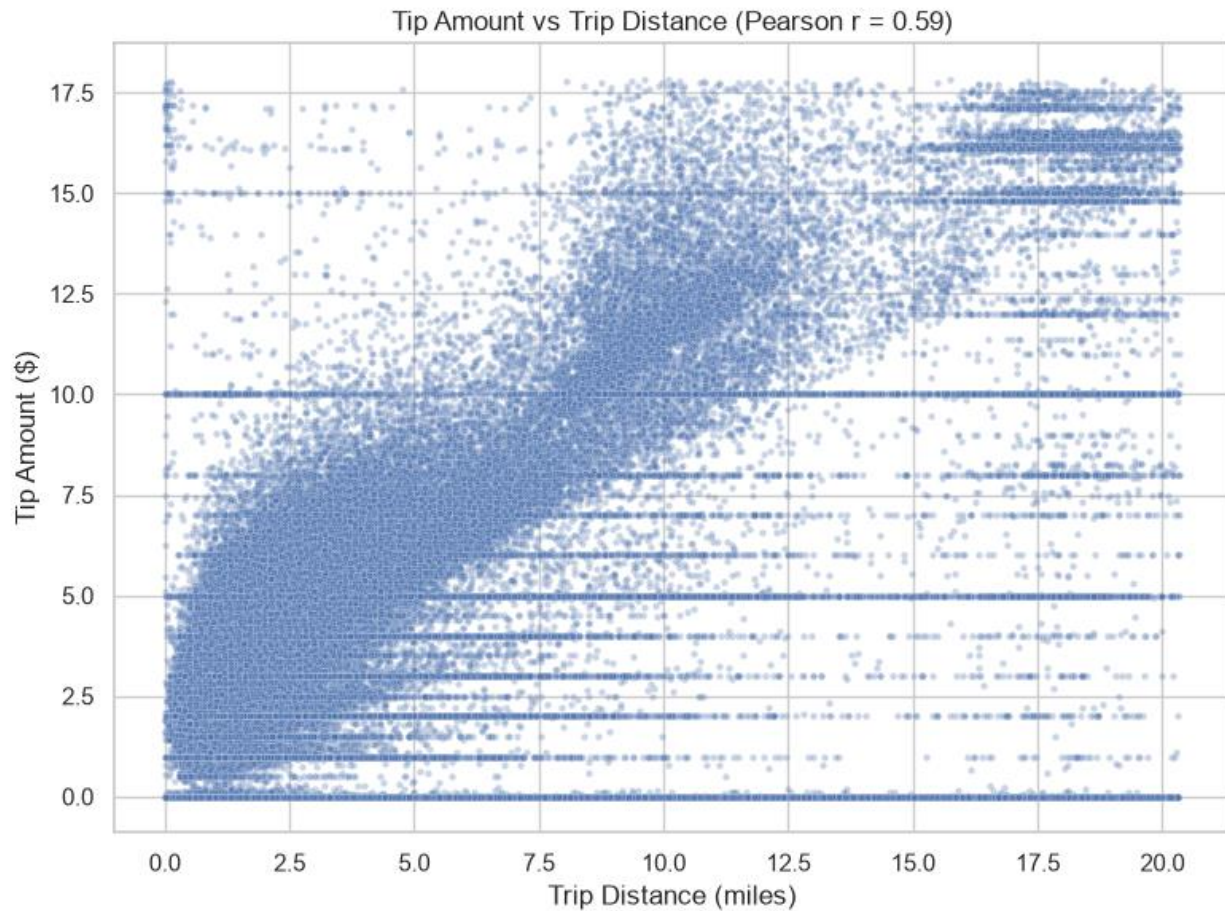
Fare amount and passenger count: The correlation between passenger_count and fare_amount was calculated. A box plot compared fare distributions across passenger-count categories. The y-axis was limited to the 99th percentile of fares to make the main distributions easier to inspect; consequently, the most extreme fares are not shown. Passenger count is discrete, so the box plot is useful for comparing groups, while the correlation should be interpreted cautiously.



The scatter plot and Pearson correlation indicate a **positive relationship** between trip distance and recorded tip amount: longer trips generally tend to have higher tips in absolute dollar terms. The relationship is not exact, since tips vary considerably between trips.

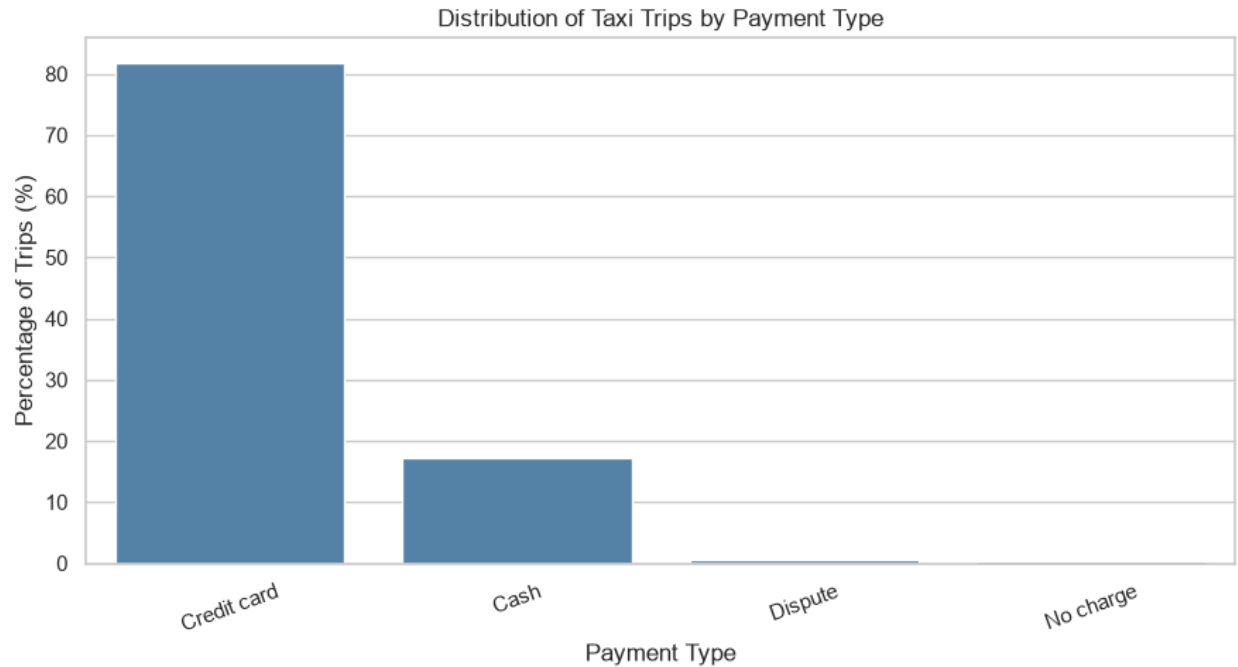
Pearson correlation: 0.5924. This measures linear association, not causation. The plot limits both axes to the 99th percentile for readability, while the correlation is calculated using all valid observations.

****Limitation:**** TLC records automatically populated credit-card tips; cash tips are not included. Therefore, the analysis describes recorded tips and may not represent total tipping behavior.



3.1.8. Analyse the distribution of different payment types

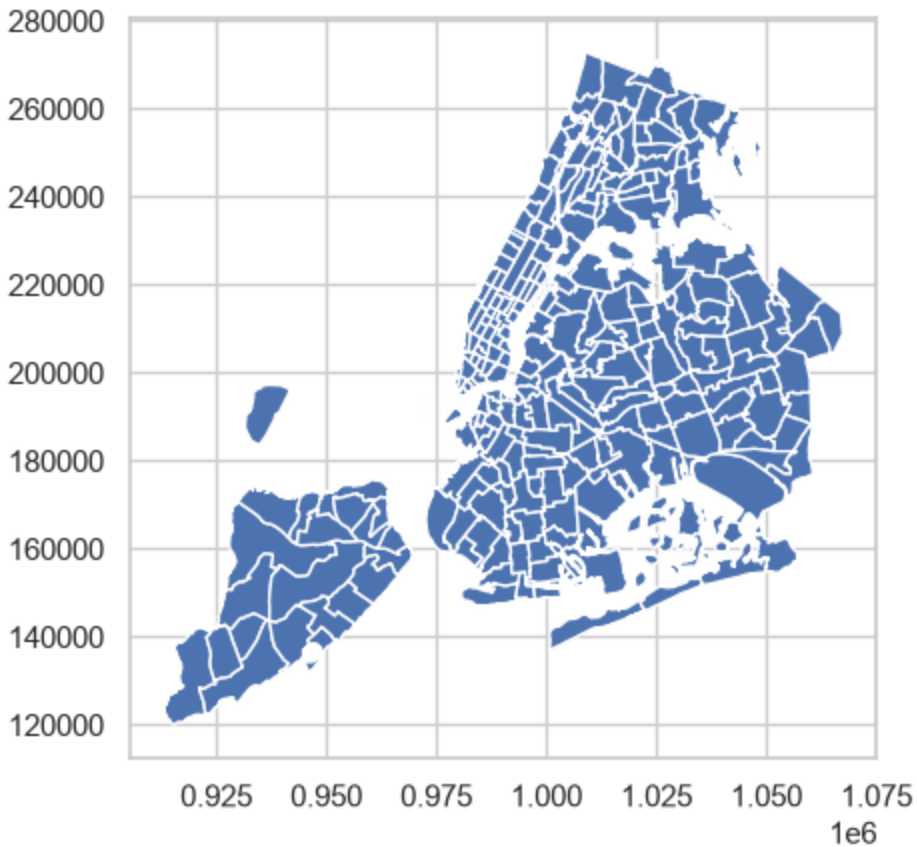
Payment type distribution: Credit card was the most common payment method, accounting for 81.75% of trips (213078 trips), followed by cash at 17.19% (44791 trips). No-charge and dispute trips represented 0.37% and 0.69%, respectively. Overall, the distribution indicates that most recorded trips were paid by credit card.



3.1.9. Load the taxi zones shapefile and display it

The taxi_zones.shp file was loaded with GeoPandas. The zone dataset provides LocationID, zone, borough and geometry, which were used for geographic analysis and mapping.

OBJECTID	Shape_Leng	Shape_Area	zone	LocationID	borough	geometry	
0	1	0.12	0.00	Newark Airport	1	EWB	POLYGON ((933100.918 192536.086, 933091.011 19...
1	2	0.43	0.00	Jamaica Bay	2	Queens	MULTIPOLYGON (((1033269.244 172126.008, 103343...
2	3	0.08	0.00	Allerton/Pelham Gardens	3	Bronx	POLYGON ((1026308.77 256767.698, 1026495.593 2...
3	4	0.04	0.00	Alphabet City	4	Manhattan	POLYGON ((992073.467 203714.076, 992068.667 20...
4	5	0.09	0.00	Arden Heights	5	Staten Island	POLYGON ((935843.31 144283.336, 936046.565 144...



3.1.10. Merge the zone data with trips data

The trip records were left-joined to the taxi-zone lookup using `pu_location_id` and `LocationID`. This adds the pickup zone's name, borough, and geometry while retaining every trip record, including trips whose location ID has no matching polygon. The merge found 2,316 unmatched pickup records (0.89%): 2,265 had placeholder ID 264 ("Unknown") and 51 had ID 265 ("Outside of NYC"). These records remain useful for aggregate counts but should be excluded from spatial maps.

LocationID	Meaning	Count	% of unmatched
264	Unknown	2,265	97.8%
265	Outside of NYC / N/A	51	2.2%

3.1.11. Find the number of trips for each zone/location ID

Trips were grouped by `pu_location_id` and counted to identify pickup locations with the highest activity. The counts were then joined to the taxi-zone lookup to add each location's zone name and borough, making the results easier to interpret. Sorting the table by `trip_count` highlights the busiest pickup zones.

This analysis covers pickup activity only; it does not count drop-offs. It uses `trip_zones_df`, which was created from `financial_df`, so the results reflect the sampled trips that passed the financial filters—not all original TLC trip records. Placeholder location IDs without matching zone details may still appear in the counts, but their zone and borough values will be missing.

Total unique pickup locations: 240

	<code>pu_location_id</code>	<code>trip_count</code>	<code>zone</code>	<code>borough</code>
0	132	13676	JFK Airport	Queens
1	237	12411	Upper East Side South	Manhattan
2	161	12147	Midtown Center	Manhattan
3	236	11164	Upper East Side North	Manhattan
4	162	9327	Midtown East	Manhattan
5	186	9147	Penn Station/Madison Sq West	Manhattan
6	138	9112	LaGuardia Airport	Queens
7	142	8785	Lincoln Square East	Manhattan
8	230	8571	Times Sq/Theatre District	Manhattan
9	163	7814	Midtown North	Manhattan

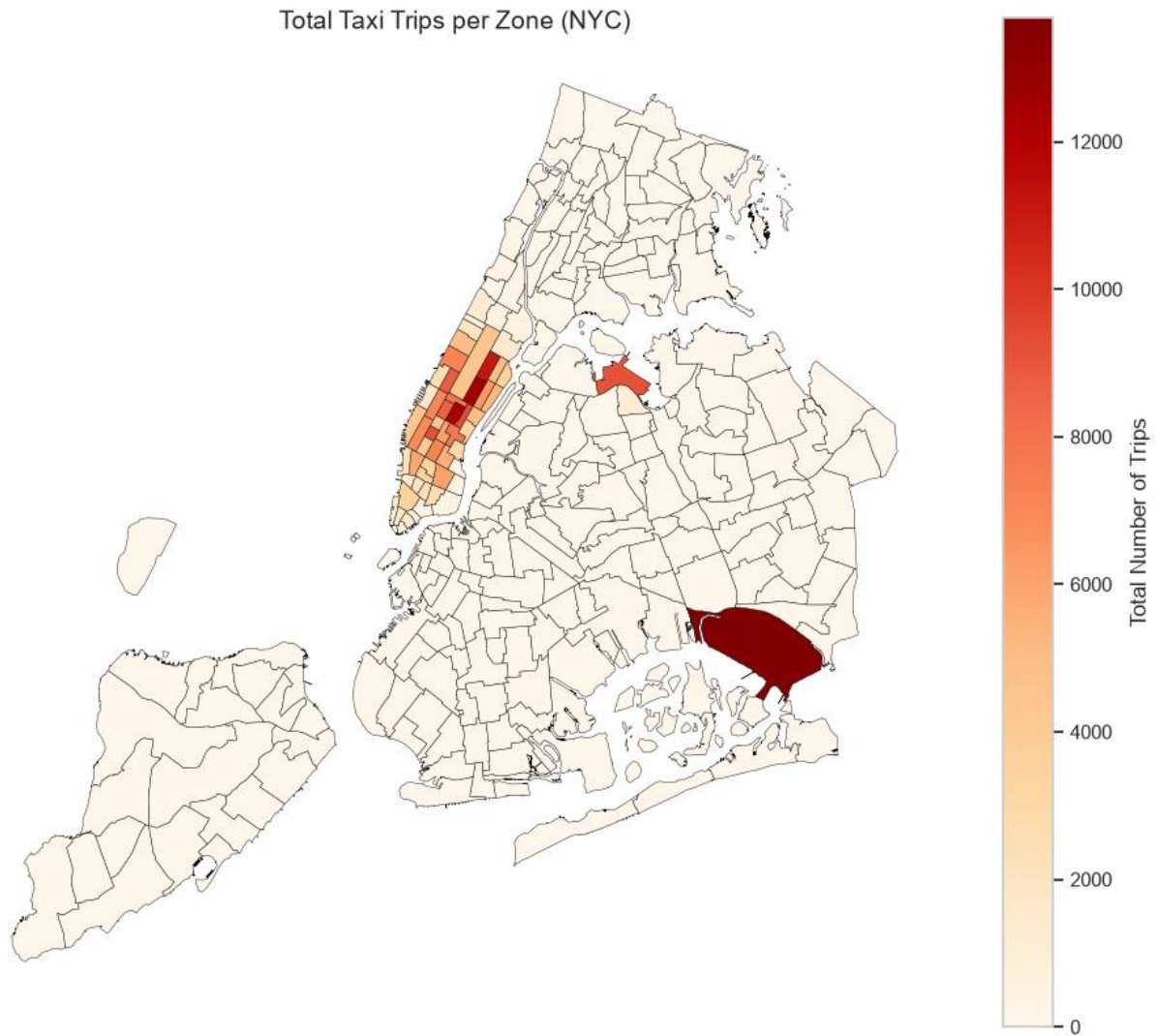
3.1.12. Add the number of trips for each zone to the zones dataframe

The pickup-trip counts were merged into the taxi-zone GeoDataFrame using the zone ID. Zones with no observed trips were assigned a count of zero, while placeholder IDs 264 and 265 were excluded because they do not represent mappable geographic areas.

3.1.13. Plot a map of the zones showing number of trips

The resulting choropleth displays pickup volume by zone: darker shading indicates more trips, and lighter shading indicates fewer.

These counts are based on financial_df, so they represent sampled trips that passed the financial filters—not all 2023 taxi trips. Use the map and sorted zone table to identify specific high- and low-volume zones; the code alone does not establish their names or counts.



3.1.14. Conclude with results

The 2023 sample reveals clear temporal and geographic patterns in NYC yellow-taxi activity. Pickups show commute and evening-demand peaks, while activity is lowest in the early morning hours. Trip volumes and revenue are broadly consistent across months and quarters, with no pronounced seasonal concentration in the analysis.

Fare amount has a strong positive relationship with trip distance and a positive, weaker relationship with trip duration. This is consistent with distance-based metered fares, with traffic and time-based charges

contributing to variation. Passenger count has little relationship with fare, as fares are primarily determined by trip distance and duration. Recorded tip amounts also tend to rise with distance (Pearson correlation reported as 0.5924), although tipping varies substantially. Because cash tips are not recorded in the TLC data, tip results should be interpreted as recorded-tip behavior rather than total tipping.

Pickup activity is concentrated in Manhattan and major airport zones, while several outer-borough zones have much lower counts. The zone map and rankings help identify where demand is concentrated and where fleet positioning may be useful. However, the mapped counts were calculated from `financial_df`, which excludes trips with zero or invalid fare, total amount, or distance; they therefore represent the filtered analysis sample, not all taxi trips. The dataset is also sampled, so the findings describe patterns in the sample rather than exact citywide totals.

Overall, the analysis supports planning for peak commute and evening demand, maintaining coverage in high-volume Manhattan and airport zones, and treating fare and tip relationships with appropriate data limitations in mind.

3.2. Detailed EDA: Insights and Strategies

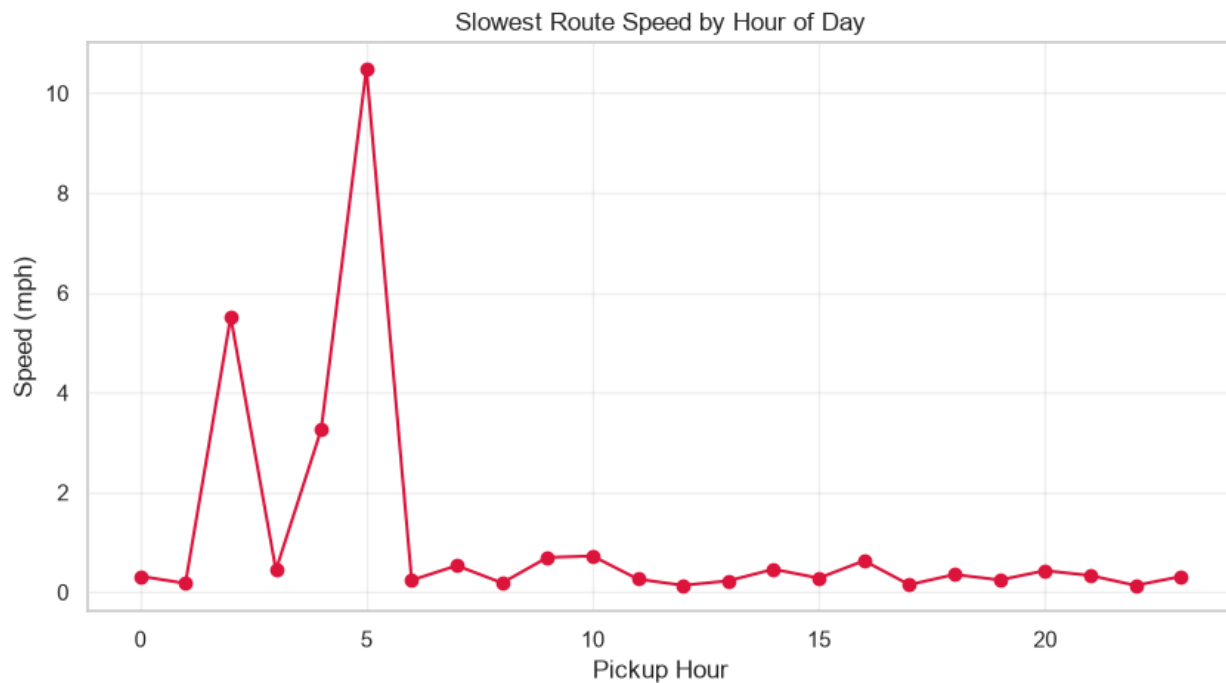
3.2.1. Identify slow routes by comparing average speeds on different routes

Routes were defined by pickup zone, drop-off zone, and pickup hour. For each route-hour combination, the analysis calculated the average trip distance, average trip duration, and number of sampled trips. To reduce the influence of sparsely observed routes, combinations with fewer than five trips were excluded.

Average speed was then estimated as average distance ÷ average duration in hours. Zone names were added for interpretation, and routes were ranked by this estimated speed. The analysis reports the 15 slowest route-hour combinations, identifies the slowest qualifying route for each hour, and plots the minimum estimated speed across the day.

This approach highlights route-hour combinations that may experience slower travel and can inform ETA buffers or routing decisions. Results should be interpreted as estimates from the filtered sample, not definitive measures of congestion: speeds can vary with traffic, and a small minimum may still be sensitive to outliers even after applying the five-trip threshold.

	pickup_zone	dropoff_zone	pickup_hour	speed_mph	trip_count
9178	Midtown South	Midtown South	22	0.13	10
3431	Greenwich Village South	Greenwich Village North	12	0.14	6
12031	UN/Turtle Bay South	UN/Turtle Bay South	17	0.15	6
7065	Meatpacking/West Village West	Flatiron	22	0.17	5
15622	Yorkville West	Yorkville West	1	0.18	7
2226	Flatiron	Midtown South	8	0.18	5
3630	Hudson Sq	TriBeCa/Civic Center	8	0.20	6
3613	Hudson Sq	Greenwich Village North	13	0.23	6
5991	Lincoln Square East	Clinton East	6	0.24	7
1678	East Village	Gramercy	19	0.24	7
12002	UN/Turtle Bay South	Sutton Place/Turtle Bay North	11	0.26	8
171	Central Park	Lenox Hill West	11	0.26	5
9202	Midtown South	Penn Station/Madison Sq West	15	0.28	10
8252	Midtown East	UN/Turtle Bay South	11	0.29	8
6660	Little Italy/NoLiTa	Gramercy	19	0.30	6

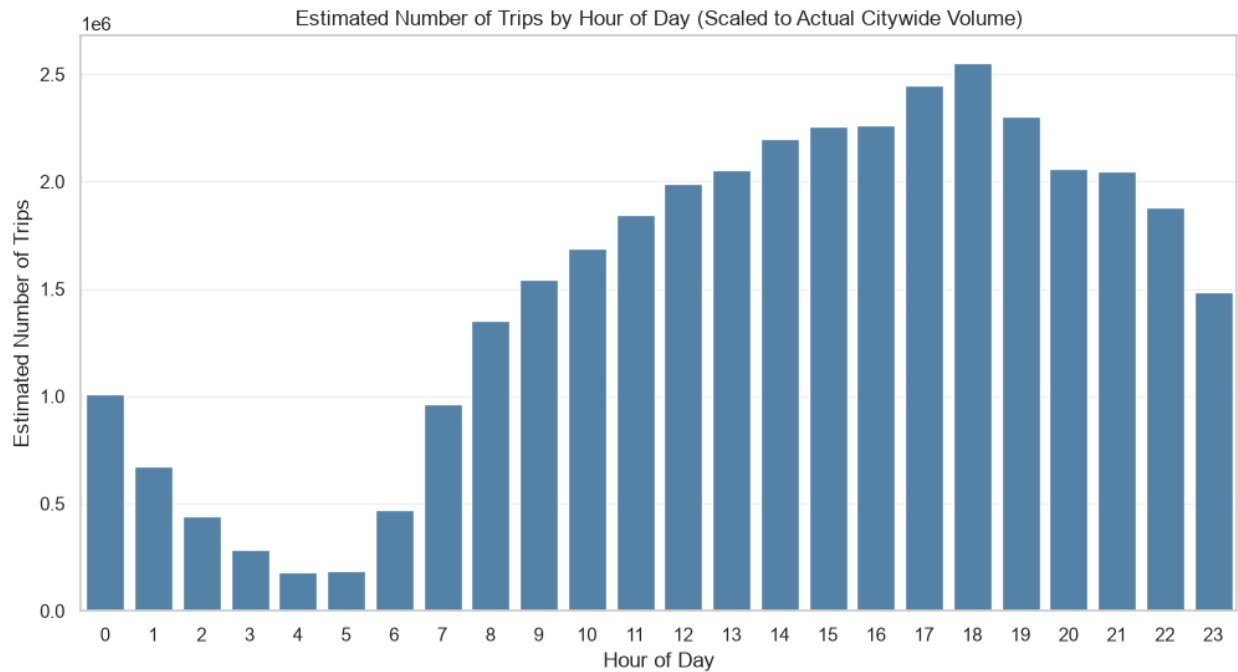


3.2.2. Calculate the hourly number of trips and identify the busy hours

The code counts trips in `financial_df` by `pickup_hour` using `value_counts()`, producing a sampled trip count for each hour from 0 to 23. It then estimates hourly volume by dividing each count by the sampling fraction, 0.0072 (0.72%). These estimates are plotted as a bar chart to compare activity across hours.

To identify the busiest hour, the code selects the hour with the largest estimated count and prints that hour and its estimated trip volume. Because every hourly count is multiplied by the same factor, scaling does not change which hour ranks highest.

Interpretation: The reported values are estimates, not direct counts of all NYC trips. Also, `financial_df` contains trips retained after financial-data filters, so the estimates describe that filtered sample and may not represent every trip in the original dataset.



3.2.3. Scale up the number of trips from above to find the actual number of trips

The code first counts trips in `financial_df` for each pickup hour. Because the dataset represents a 0.72% sample, each hourly sample count is divided by 0.0072—equivalent to multiplying by approximately 138.89—to estimate the corresponding trip volume.

The estimated hourly counts are rounded to whole trips, sorted from highest to lowest, and the top five are displayed. These are scaled estimates, not direct counts of all NYC trips; they are based on `financial_df`, which contains trips retained after financial filters.

```
Actual (scaled) number of trips in the five busiest hours:
```

```
Hour 18:00 → 2,551,806 trips  
Hour 17:00 → 2,447,083 trips  
Hour 19:00 → 2,301,250 trips  
Hour 16:00 → 2,262,917 trips  
Hour 15:00 → 2,259,028 trips
```

3.2.4. Compare hourly traffic on weekdays and weekends

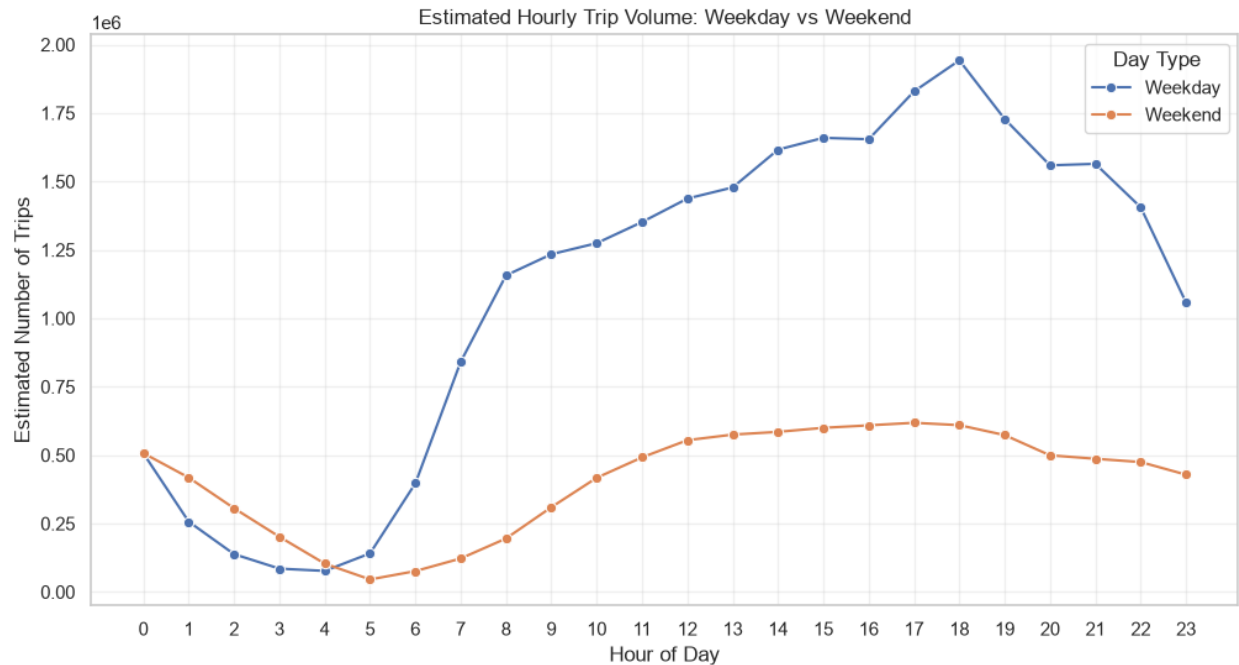
The code classifies trips as Weekday (Monday–Friday) or Weekend (Saturday–Sunday), groups the filtered trips in `financial_df` by day type and pickup hour, and scales each sampled count by dividing by 0.0072. A line chart then compares estimated hourly trip volumes; the code also identifies the busiest and quietest hour for each day type.

The recorded results show different demand profiles:

- **Weekdays:** Traffic rises sharply during the morning commute and reaches its highest level around 6 PM. It then declines through the evening.
- **Weekends:** Traffic is lowest around 5 AM, increases through the day, and forms a broader plateau from roughly 1 PM to 6 PM. Late-night activity remains comparatively strong, consistent with nightlife demand.

These patterns can guide scheduling: concentrate weekday availability around commute peaks, while maintaining broader afternoon and evening coverage on weekends.

Interpretation note: The chart compares total trips accumulated across all weekdays and all weekend days in the dataset—not an average weekday against an average weekend day. Since there are five weekdays but only two weekend days per week, compare normalized daily averages for a fairer day-type comparison. The estimates also reflect `financial_df`'s positive fare, total amount, and distance filters, rather than every taxi record.



```

Weekday:
  Busiest hour → 18:00 with ~1,942,361 trips
  Quietest hour → 04:00 with ~76,944 trips

Weekend:
  Busiest hour → 17:00 with ~618,194 trips
  Quietest hour → 05:00 with ~45,694 trips

```

3.2.5. Identify the top 10 zones with high hourly pickups and drops

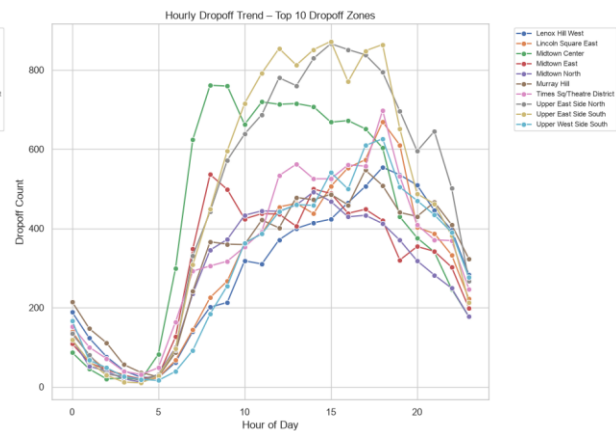
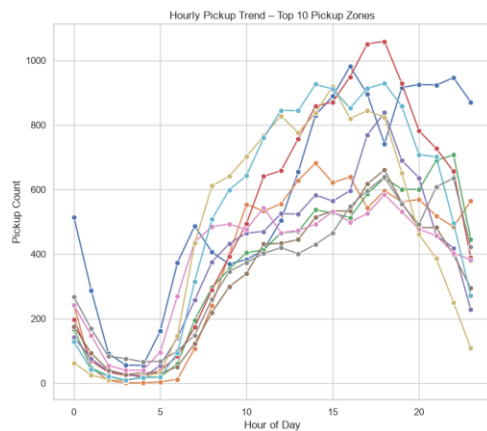
The code identifies the 10 busiest pickup zones by counting records in `financial_df` for each `pu_location_id`, sorting those counts in descending order, and selecting the top 10. It then joins the selected zone IDs to `taxi_zones` to display zone names and boroughs.

For those zones, the code groups trips by pickup zone and pickup_hour and plots the hourly counts as separate lines. This shows how pickup activity in the busiest zones changes over the 24-hour day and helps compare their demand profiles.

Interpretation: The chart can reveal whether high-volume zones share similar daily peaks or have distinct demand patterns. These patterns can inform where and when to position vehicles. However, the code selects zones based on their overall pickup totals, not on the highest count in any single hour. It also plots counts from the sampled, financially filtered `financial_df`; it does not scale them to estimate citywide totals.

Top 10 Pickup Zones:

	zone	borough	pickup_count
116	JFK Airport	Queens	13676
214	Upper East Side South	Manhattan	12411
144	Midtown Center	Manhattan	12147
213	Upper East Side North	Manhattan	11164
145	Midtown East	Manhattan	9327
167	Penn Station/Madison Sq West	Manhattan	9147
122	LaGuardia Airport	Queens	9112
126	Lincoln Square East	Manhattan	8785
207	Times Sq/Theatre District	Manhattan	8571
146	Midtown North	Manhattan	7814



3.2.6. Find the ratio of pickups and dropoffs in each zone

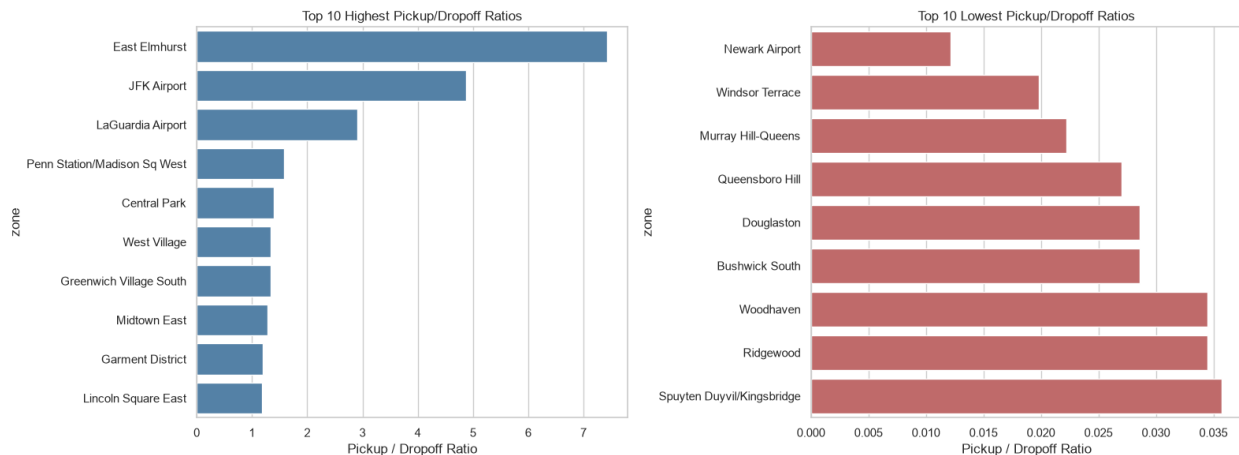
To compare pickup and dropoff activity across zones, the code first counts trips in `financial_df` by pickup zone (`pu_location_id`) and dropoff zone (`do_location_id`) separately. It then combines the counts using an outer join on the zone ID, so zones appearing in only one count table are retained. Missing pickup or dropoff counts are filled with zero.

To reduce unstable results from very low-volume zones, the code keeps only zones with at least 20 combined pickups and dropoffs. It then calculates:

Pickup/dropoff ratio = pickup count ÷ dropoff count

For zones with no recorded dropoffs, the code substitutes 0.5 for the denominator to avoid division by zero. Finally, it sorts the ratios to show the 10 highest and 10 lowest zones and plots both groups. A high ratio indicates relatively more pickups than dropoffs; a low ratio indicates relatively more dropoffs than pickups.

Interpretation and caveats: These are counts from the financially filtered sample, not the full trip dataset. The 0.5 denominator is a computational adjustment, not an observed dropoff count, so zones with zero dropoffs can receive artificially large ratios. Also, the outer join uses zone names and boroughs from the pickup-count table; a zone appearing only in the dropoff counts may therefore have missing labels. The code output is needed to state which zones ranked highest or lowest.



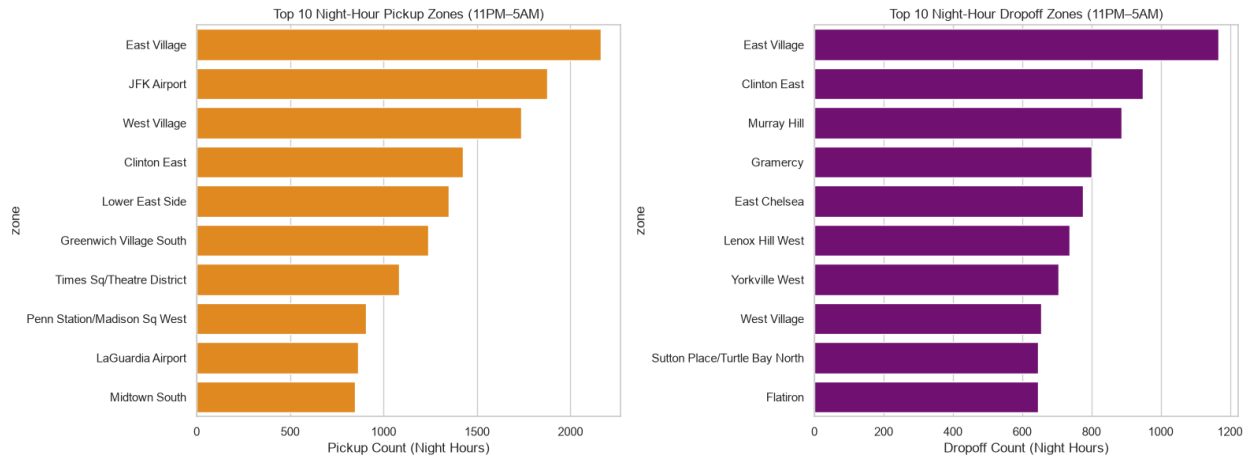
3.2.7. Identify the top zones with high traffic during night hours

The analysis uses `financial_df`, which contains trips that passed the financial filters. It defines night hours as 23:00 through 04:59 using the hour values [23, 0, 1, 2, 3, 4], then selects trips whose pickup hour falls in that range.

For those trips, it separately groups records by pickup-zone ID and dropoff-zone ID, counts trips in each zone, and joins the counts to `taxi_zones` to add zone names and boroughs. It sorts each result by count, selects the 10 highest-volume zones, prints the tables, and plots pickup and dropoff counts in separate bar charts.

Interpretation: The pickup ranking shows where the selected nighttime trips began; the dropoff ranking shows where those same trips ended. These rankings can help identify nighttime areas for fleet positioning and dispatch planning.

Important limitation: The dropoff counts are based on the trip's pickup hour, not the actual dropoff time. To measure dropoffs occurring between 11 PM and 5 AM, the filter should use `tpdp_dropoff_datetime` instead. Also, the code's interval ends at 04:59—hour 5 is not included—and it reports counts from the sample without scaling them to estimated citywide totals. The specific top zones depend on the cell's output.



3.2.8. Find the revenue share for nighttime and daytime hours

The analysis classified each trip in `financial_df` by its pickup hour. The code defines nighttime as hours 23:00–05:59 ([23, 0, 1, 2, 3, 4, 5]); all other pickup hours are classified as daytime.

It then sums `total_amount` for each period, using it as the revenue measure, and calculates each period’s share of the combined revenue:

Revenue share (%) = period total revenue ÷ total revenue across both periods × 100

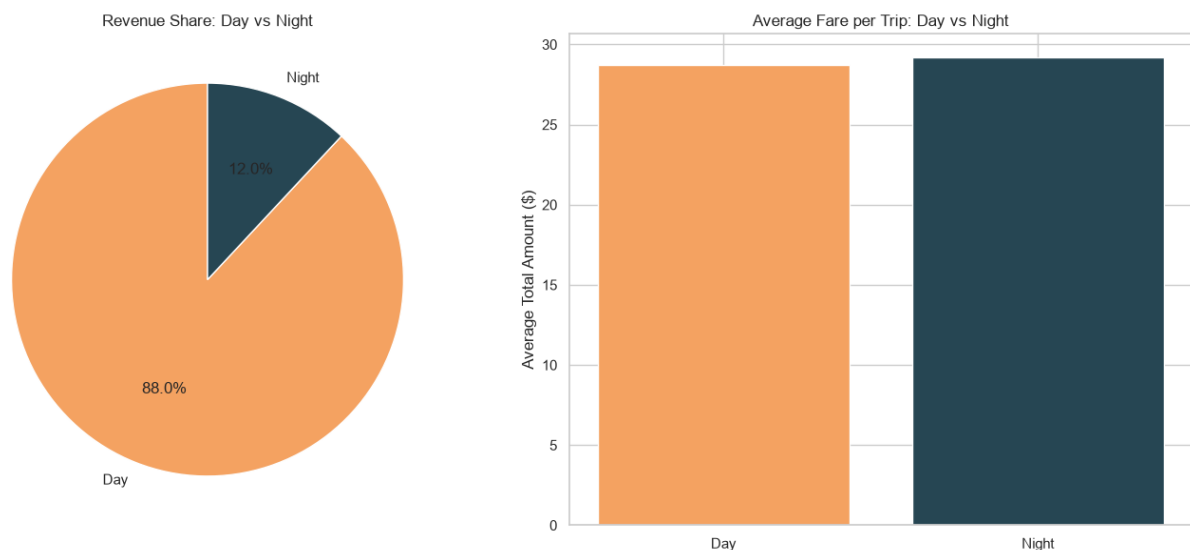
For context, the code also calculates trip counts, each period’s share of trips, and average `total_amount` per trip. It prints these measures in a summary table and visualizes the revenue shares in a pie chart and average amounts in a bar chart.

Interpretation: Comparing revenue share with trip share shows whether nighttime or daytime trips contribute a larger proportion of the sampled revenue relative to their trip volume. The average-per-trip measure provides additional context. The notebook output is needed to report the actual percentages and determine which period contributes more.

Caveats: The analysis uses pickup time, so trips are assigned by when they began—not when they ended. The code includes the full 5 AM hour, despite the section heading’s “to 5AM” wording. Also, results come from `financial_df`, which excludes trips with non-positive fare, total amount, or distance. Because the data is sampled, the shares describe the analyzed sample; a common scaling factor would not change the revenue-share percentages.

	time_period	trip_count	trip_share_pct	total_revenue	revenue_share_pct	\
0	Day	229864	88.20	6,606,780.83	88.03	
1	Night	30766	11.80	898,139.02	11.97	

	avg_fare_per_trip
0	28.74
1	29.19



3.2.9. For the different passenger counts, find the average fare per mile per passenger

To compare the fare burden across passenger counts, the analysis first filtered `financial_df` to retain trips with positive trip distance, positive fare, and positive passenger count. It then calculated each trip's fare per mile as `fare_amount / trip_distance`.

Next, trips were grouped by `passenger_count`, and the mean fare per mile was calculated for each group. The code estimated fare per mile per passenger by dividing that group average by the passenger count:

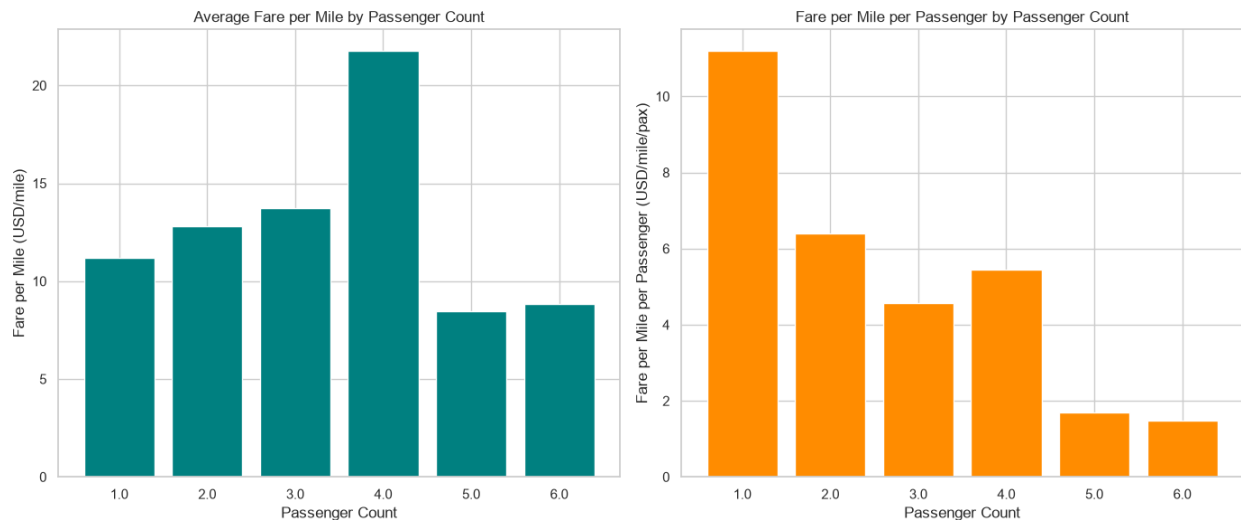
Average fare per mile per passenger = mean fare per mile for the passenger-count group ÷ passenger count

Groups with fewer than 20 trips were excluded from the displayed results to reduce the influence of sparse data. The results were printed in a summary table and visualized with bar charts showing average fare per mile and average fare per mile per passenger.

This is an equal-share estimate: it assumes the trip fare is divided evenly among passengers. It does not measure what each passenger actually paid. Since the notebook output is not provided here, the specific values

and which passenger-count group had the highest or lowest estimate cannot be stated.

	passenger_count	trip_count	avg_fare_per_mile	fare_per_mile_per_passenger
0	1.00	199772	11.20	11.20
1	2.00	39904	12.80	6.40
2	3.00	9729	13.71	4.57
3	4.00	5514	21.77	5.44
4	5.00	3427	8.46	1.69
5	6.00	2284	8.85	1.48



3.2.10. Find the average fare per mile by hours of the day and by days of the week

This analysis examined how the average fare per mile varied by pickup hour and day of the week.

First, the code created `valid_trips` by retaining trips with a positive `trip_distance` and positive `fare_amount`. It calculated fare per mile for each trip as:

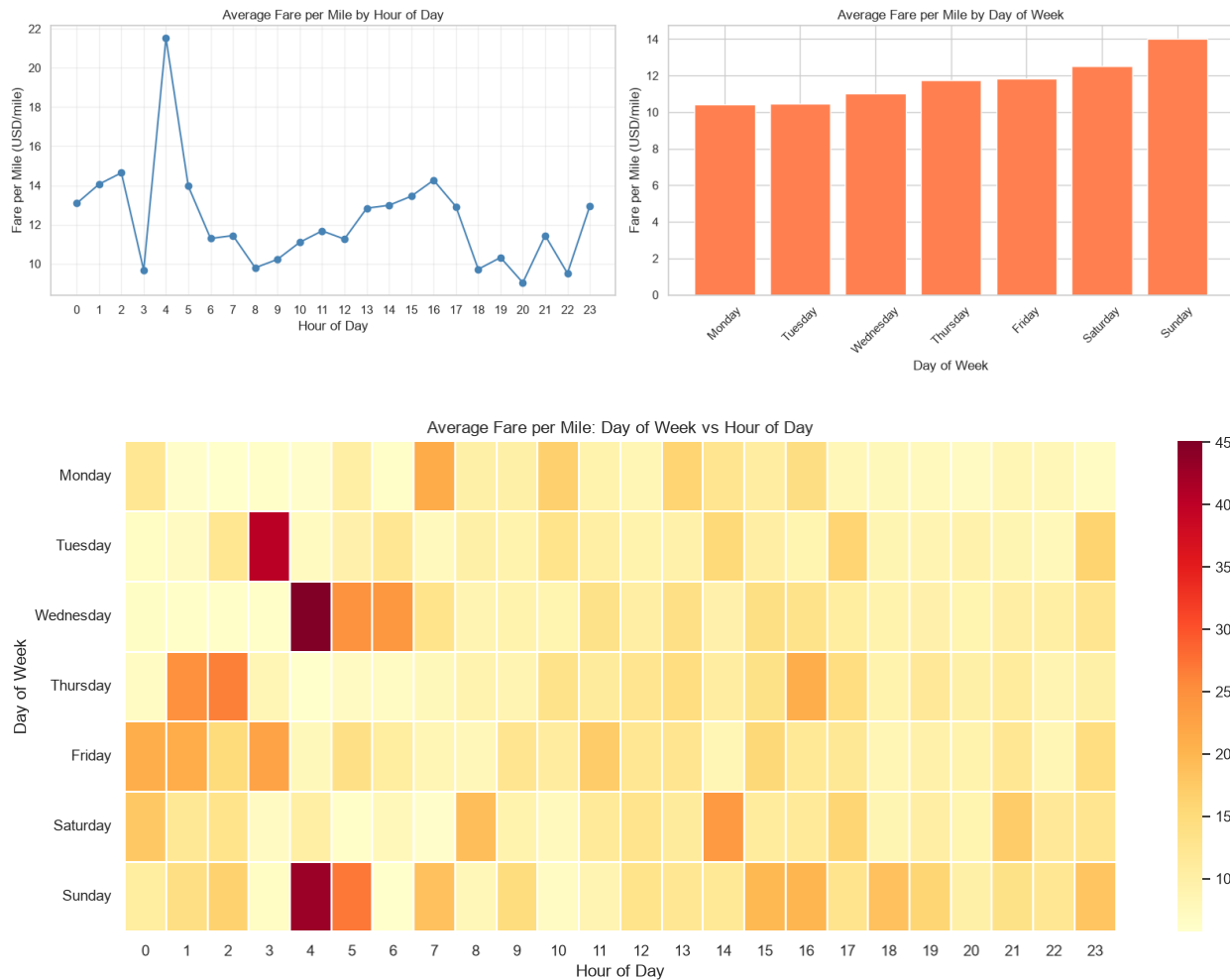
Fare per mile = fare amount ÷ trip distance

The trip-level values were then averaged in three ways:

- By hour: grouped by `pickup_hour` (0–23) to find the mean fare per mile for each hour.
- By day: grouped by the pickup day of the week, ordered Monday through Sunday, to find the mean for each day.

- By hour and day: grouped by both day and hour, then arranged in a heatmap to show how the hourly pattern differs across weekdays and weekends.

The line plot displays hourly averages, the bar chart compares daily averages, and the heatmap combines both dimensions. Since the code calculates the mean of individual trip-level fare-per-mile values, the result is not the total fare divided by total distance for each group. Very short trips can produce high fare-per-mile values and influence the averages. The notebook output is not included here, so specific peak hours, days, or fare-per-mile amounts cannot be reported.

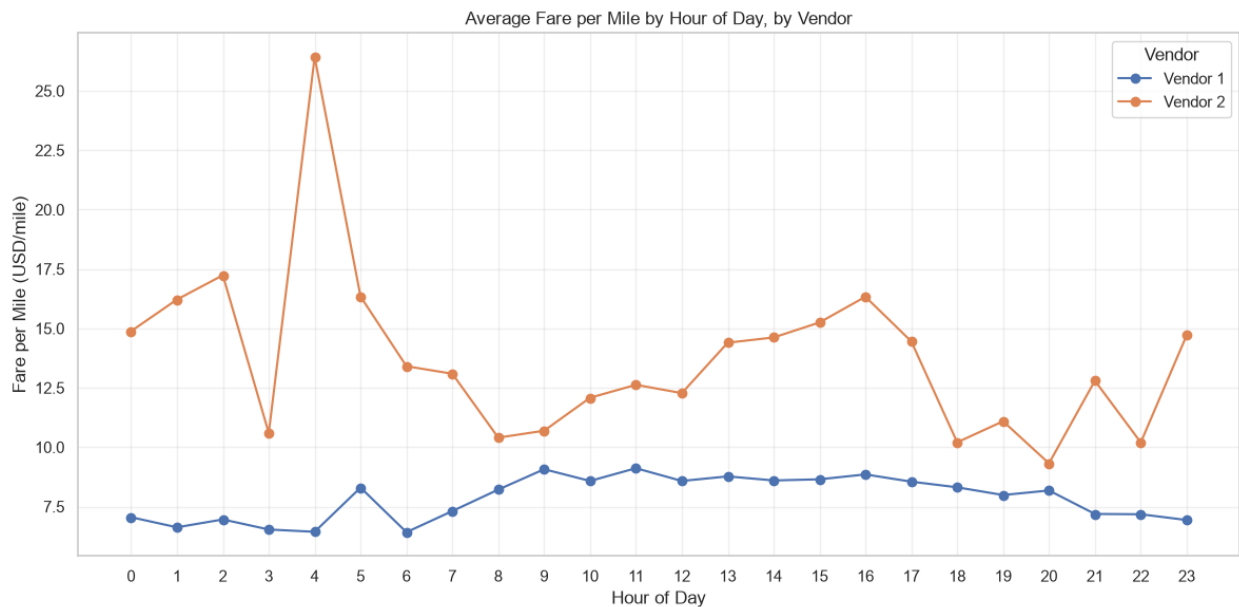


3.2.11. Analyse the average fare per mile for the different vendors

To compare vendors, the analysis used `valid_trips`, which includes trips with positive distance and positive fare. It calculated each trip's fare per mile as $\text{fare amount} \div \text{trip distance}$, then derived the pickup hour from the pickup timestamp.

The code grouped trips by vendor and pickup hour to calculate the mean fare per mile for each vendor-hour combination. A line chart compared vendors across the 24 hours of the day. It also calculated and printed each vendor's overall average fare per mile, along with the hourly averages.

This approach shows whether recorded fare-per-mile patterns differ between vendors or change by time of day. However, the notebook excerpt does not include the cell's output, so it does not support identifying which vendor had the higher average or naming peak hours. These are averages of trip-level ratios, so short trips or unusually high fare-per-mile trips may affect the results. The comparison also does not apply a minimum trip-count threshold for each vendor-hour group.



3.2.12. Compare the fare rates of different vendors in a distance-tiered fashion

To compare vendor fare rates fairly across trip lengths, trips with a positive distance were divided into three tiers: short (≤ 2 miles), medium (> 2 to 5 miles, despite the label “2–5 mi”), and long (> 5 miles). For each trip, fare per mile was calculated as $\text{fare_amount} \div \text{trip_distance}$.

The analysis grouped trips by distance tier and vendor, then calculated the mean fare per mile, median fare per mile, and trip count for each group. A pivot table and grouped bar chart compared the average rates across tiers and vendors. Boxplots showed the distributions within each

tier; showfliers=False hides extreme points from the visual only, not from the summary statistics.

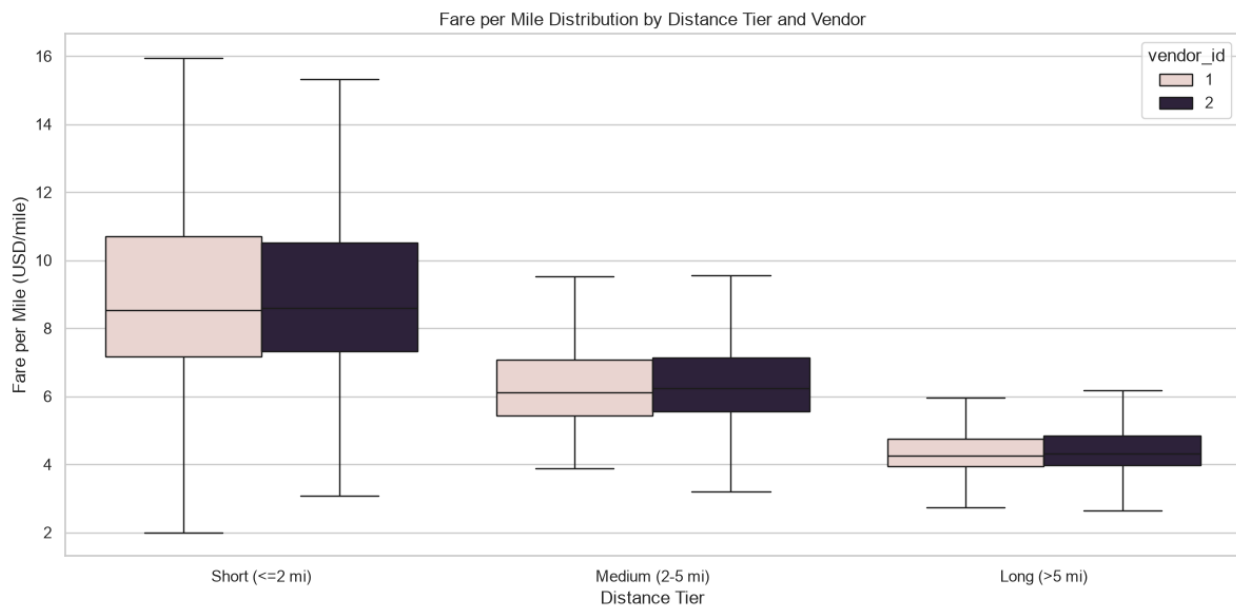
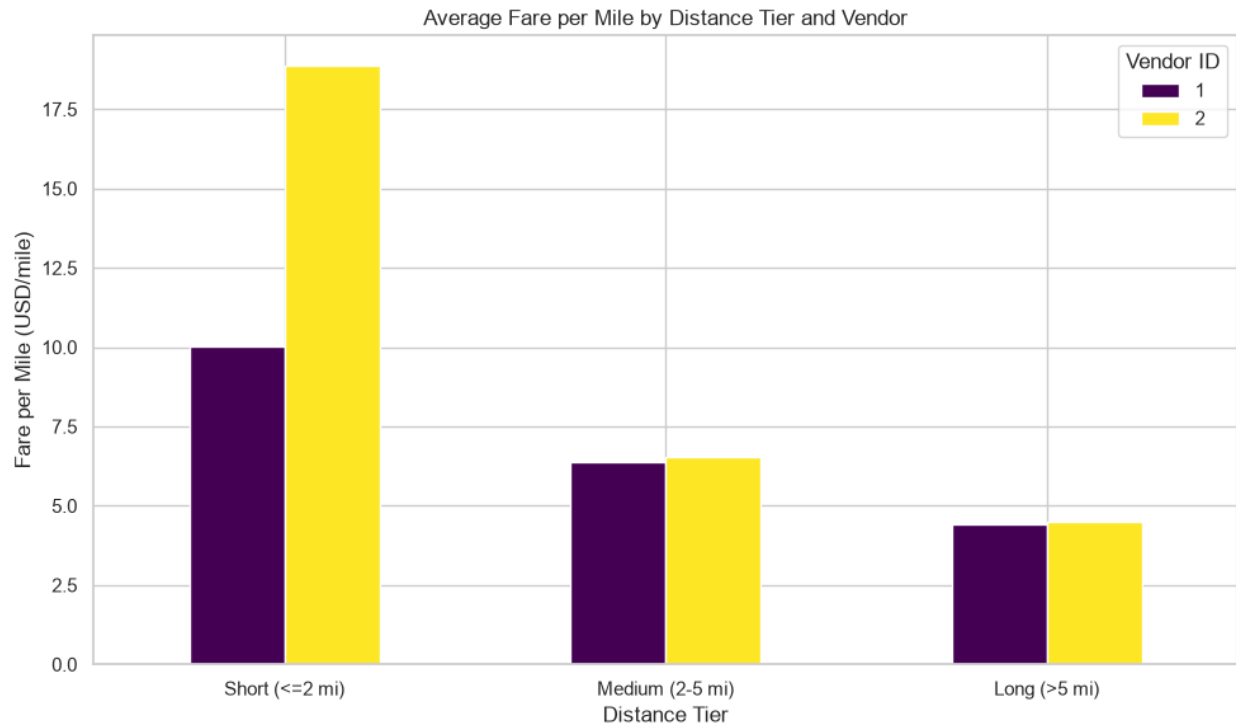
Segmenting by distance helps make vendor comparisons less affected by differences in trip-length mix. The mean reflects the average trip-level fare-per-mile ratio, while the median provides a less outlier-sensitive comparison. Because the dataset is sampled, the reported trip counts describe the sample; the mean and median rates themselves are not scaled.

The code shown does not include the resulting table or plots, so it does not establish which vendor had the higher rate in any tier. Also, trips with zero fares may remain in the data and affect the averages.

```
distance_tier vendor_id avg_fare_per_mile median_fare_per_mile \
0 Short (<=2 mi) 1 10.01 8.53
1 Short (<=2 mi) 2 18.87 8.59
2 Medium (2-5 mi) 1 6.40 6.10
3 Medium (2-5 mi) 2 6.55 6.24
4 Long (>5 mi) 1 4.43 4.27
5 Long (>5 mi) 2 4.50 4.31
```

```
trip_count
0 39657
1 105577
2 17730
3 52760
4 10666
5 34240
```

```
Pivoted view (avg fare per mile):
vendor_id 1 2
distance_tier
Short (<=2 mi) 10.01 18.87
Medium (2-5 mi) 6.40 6.55
Long (>5 mi) 4.43 4.50
```



3.2.13. Analyse the tip percentages

Tip percentage was calculated for trips with positive fare and distance as:

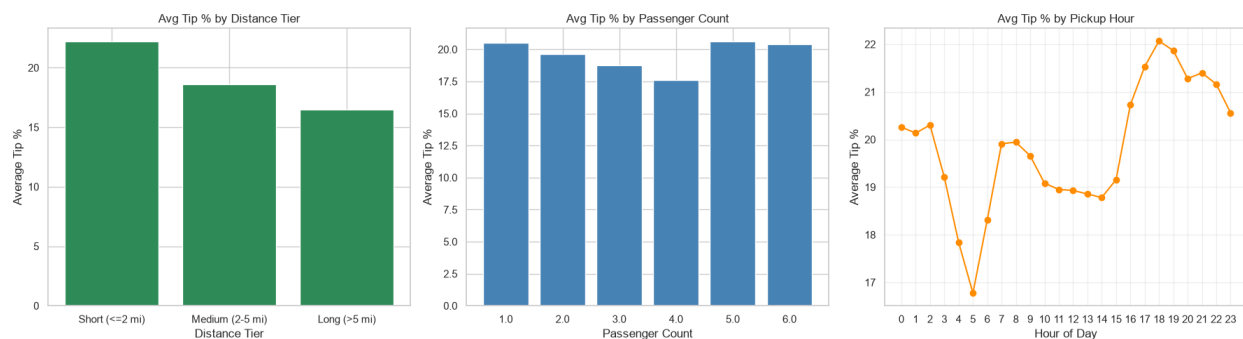
$$\text{tip_amount} \div \text{fare_amount} \times 100$$

Records with percentages below 0% or above 100% were excluded to limit the effect of invalid or extreme values. The remaining trips were

grouped in three ways: by distance tier (up to 2 miles, over 2 to 5 miles, and over 5 miles), passenger count, and pickup hour. The mean tip percentage was calculated for each group and visualized with bar and line charts. Passenger-count groups with fewer than 20 trips were excluded from that comparison to reduce the influence of small samples.

The code also compared average tip percentage by payment type. This provides context for interpreting low observed tips: TLC records automatically populated credit-card tips, while cash tips are not recorded. Consequently, cash trips can appear to have low or zero tip percentages even when a cash tip was given. Tip-percentage differences—particularly by pickup hour—should therefore be interpreted as recorded-tip patterns, not necessarily differences in actual tipping behavior.

The code produces the summary tables and plots needed to identify which groups have lower averages, but their results are not shown here; specific group comparisons cannot be stated without those outputs.



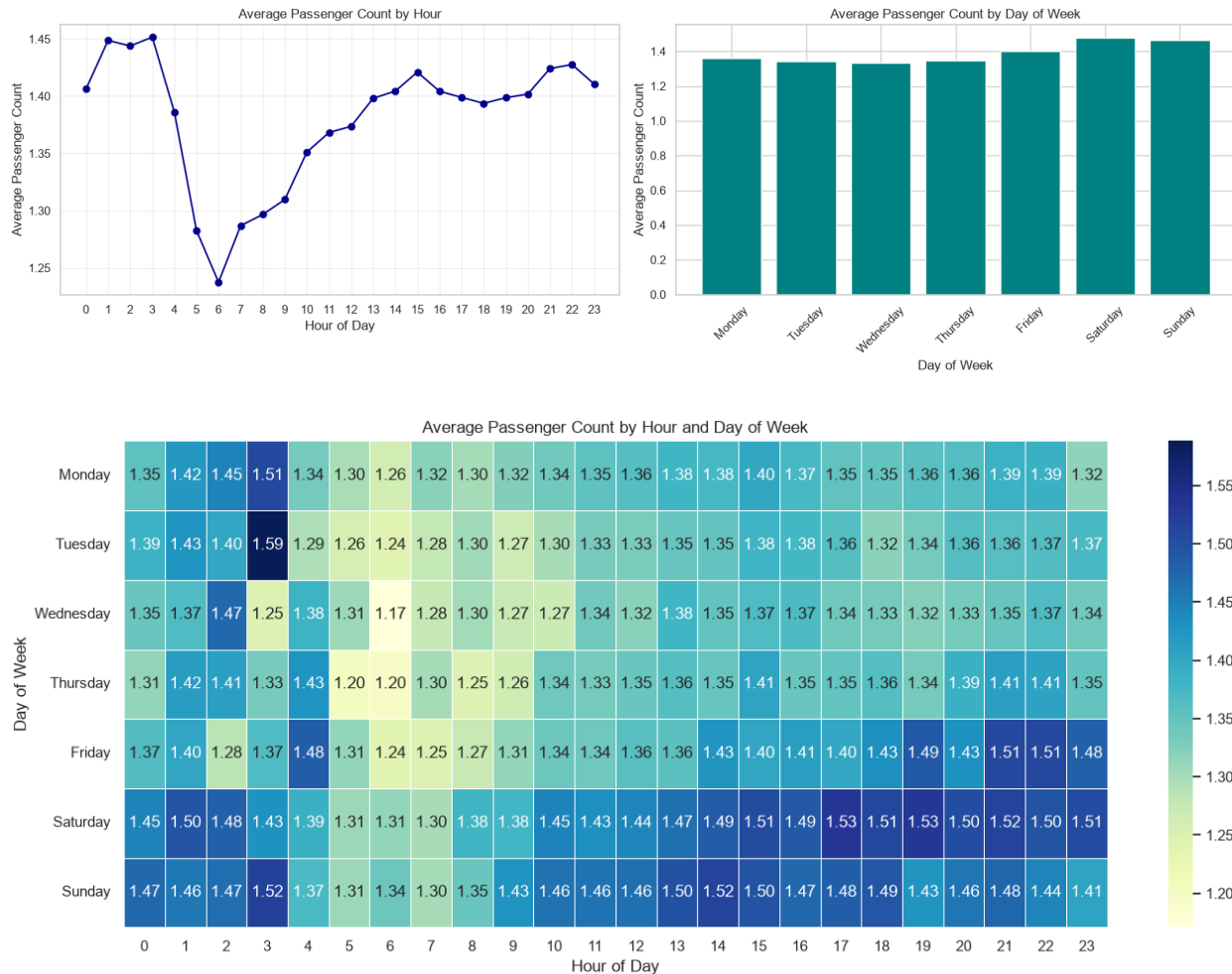
3.2.14. Analyse the trends in passenger count

Passenger-count trends were analyzed using the cleaned taxi-trip sample. The pickup day was represented by its weekday name and ordered from Monday to Sunday. The analysis then calculated the average recorded passenger count for each pickup hour, for each day of the week, and for every hour–day combination.

A line chart shows average passenger count across the 24 hours, a bar chart compares daily averages, and a heatmap displays the combined hour-and-day pattern. The heatmap helps identify times when trips tend to have higher or lower recorded occupancy.

These charts describe average occupancy per trip, not the number of trips or total passengers carried. Passenger counts are driver-entered and may include imputed values from the cleaning stage, so small differences should be interpreted cautiously. Specific peak or low-

occupancy periods should be reported from the cell's printed tables and visualizations; those results were not included in the provided excerpt.



3.2.15. Analyse the variation of passenger counts across zones

Passenger-count variation was analyzed by pickup zone using the cleaned taxi-trip sample. For each pickup location ID, the code calculated the mean passenger count and the number of trips. It then retained zones with at least 20 trips to reduce the influence of averages based on very small samples. Zone IDs were matched to zone names and boroughs, and the highest and lowest average passenger-count zones were listed for comparison.

A choropleth map was also created by joining each zone's average passenger count to the taxi-zone GeoDataFrame. Zones 264 and 265 were excluded from the map because they are placeholder locations without valid geographic areas; zones without usable data were shown in grey.

The analysis compares average passengers per trip, not the total passenger volume or number of trips in each zone. Passenger counts are driver-entered, and missing or zero values were replaced with the median during cleaning, so averages should be interpreted with that limitation in mind. The code identifies the zones with the highest and lowest averages, but their names and values should be taken from the cell's output rather than inferred from the method alone.

Top 10 zones by average passenger count:

	zone	borough	avg_passenger_count	\
3	Battery Park	Manhattan	1.77	
2	Baisley Park	Queens	1.70	
21	DUMBO/Vinegar Hill	Brooklyn	1.67	
34	Flushing Meadows-Corona Park	Queens	1.67	
66	Old Astoria	Queens	1.65	
97	Williamsburg (North Side)	Brooklyn	1.58	
100	World Trade Center	Manhattan	1.55	
54	Long Island City/Hunters Point	Queens	1.53	
83	Times Sq/Theatre District	Manhattan	1.51	
46	JFK Airport	Queens	1.50	

trip_count

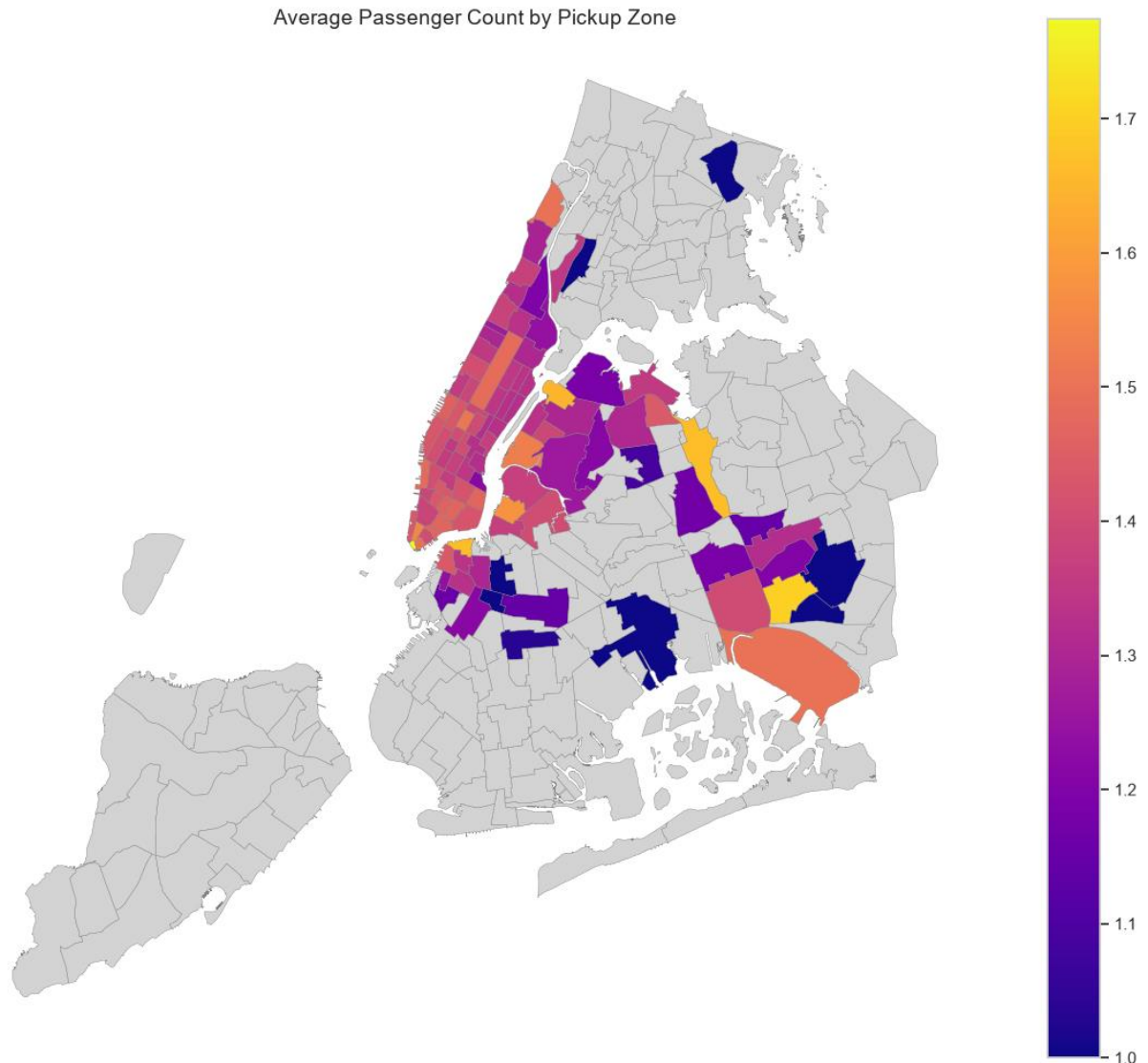
3	102
2	90
21	75
34	54
66	37
97	64
100	1343
54	79
83	8571
46	13676

Bottom 10 zones by average passenger count:

	zone	borough	avg_passenger_count	\
17	Co-Op City	Bronx	1.00	
78	Springfield Gardens North	Queens	1.00	
27	East New York	Brooklyn	1.00	
23	East Concourse/Concourse Village	Bronx	1.00	
73	Saint Albans	Queens	1.00	
15	Clinton Hill	Brooklyn	1.00	
70	Prospect Heights	Brooklyn	1.00	
69	Prospect-Lefferts Gardens	Brooklyn	1.03	
30	Elmhurst	Queens	1.09	
19	Crown Heights North	Brooklyn	1.15	

trip_count

17	24
78	25
27	26
23	20
73	28
15	24
70	21
69	29
30	23
19	34



3.2.16. Analyse the pickup/dropoff zones or times when extra charges are applied more frequently.

The analysis examined how often surcharge fields were non-zero and how their application varied by time and location. It used a copy of the cleaned trip data so the surcharge analysis would not alter the main dataset.

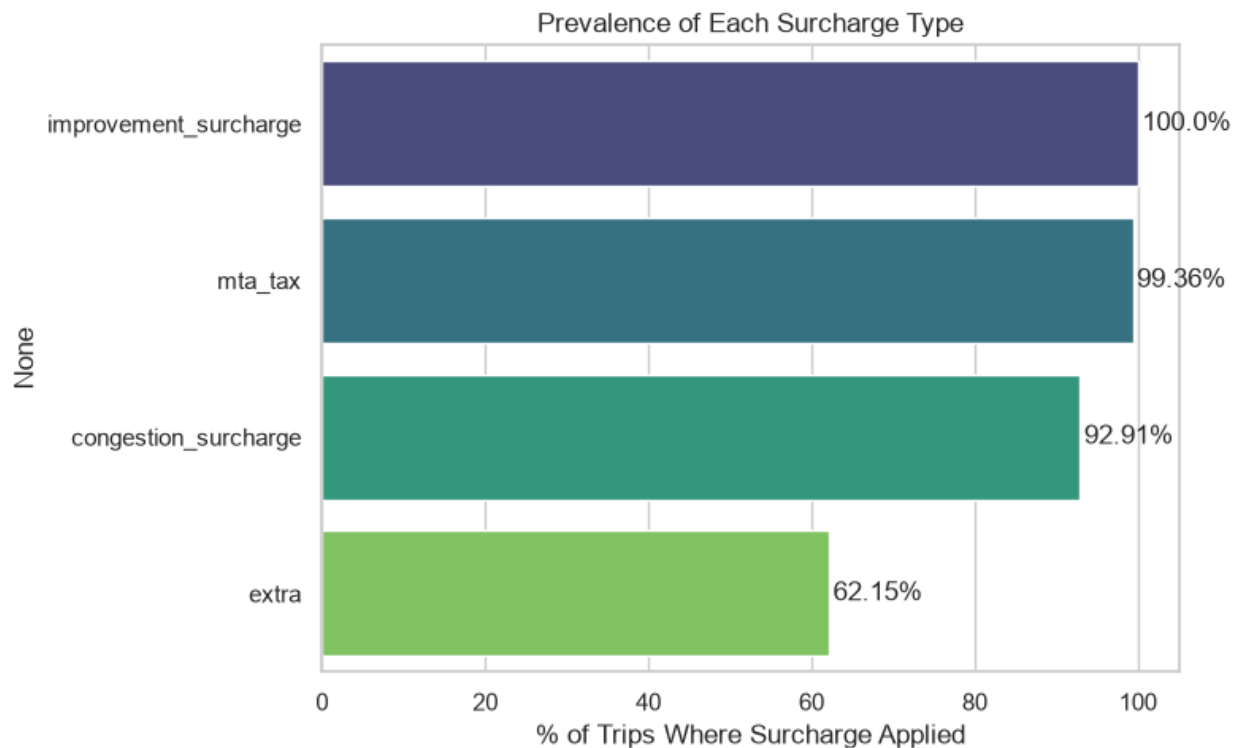
Overall prevalence (cell 148): The code selected whichever of extra, mta_tax, improvement_surcharge, congestion_surcharge, and airport_fee were present. For each field, it calculated the percentage of trips where

the value was non-zero and plotted the results. This provided an overall comparison of how frequently each listed charge appeared.

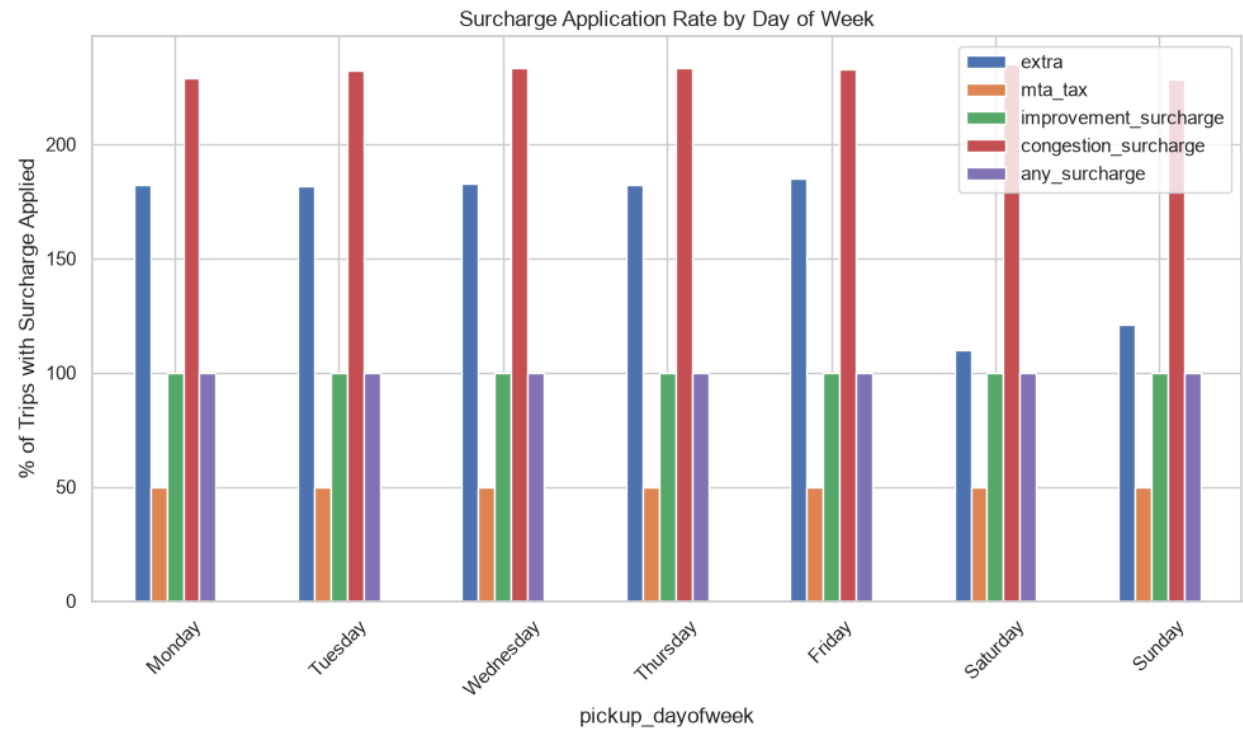
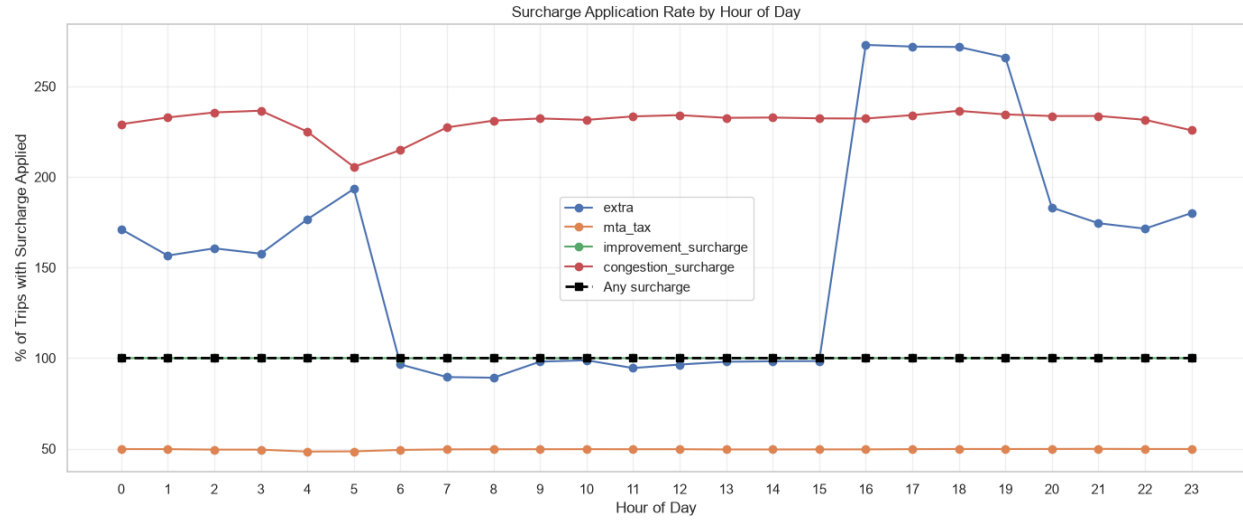
```
Surcharge columns found: ['extra', 'mta_tax', 'improvement_surcharge',  
'congestion_surcharge']
```

```
Percentage of trips with each surcharge applied:
```

```
improvement_surcharge  100.00  
mta_tax                99.36  
congestion_surcharge   92.91  
extra                  62.15  
Name: pct_trips_applied, dtype: float64
```



Variation by time (cells 150 and 151): The code created an any_surcharge flag for trips with at least one non-zero value among the selected fields. It then calculated the percentage of trips with each charge—and with any charge—by pickup hour and by day of the week. Line and bar charts were used to compare these application rates across time periods.



...	extra	mta_tax	improvement_surcharge	congestion_surcharge	\
pickup_dayofweek					
Monday	182.38	49.67	99.96	229.15	
Tuesday	181.94	49.74	99.96	232.58	
Wednesday	182.65	49.72	99.95	233.43	
Thursday	182.01	49.68	99.98	233.16	
Friday	184.86	49.68	99.97	232.83	
Saturday	110.13	49.68	99.98	235.32	
Sunday	121.14	49.59	99.96	228.39	
	any_surcharge				
pickup_dayofweek					
Monday		100.00			
Tuesday		100.00			
Wednesday		100.00			
Thursday		100.00			
Friday		99.99			
Saturday		100.00			
Sunday		100.00			

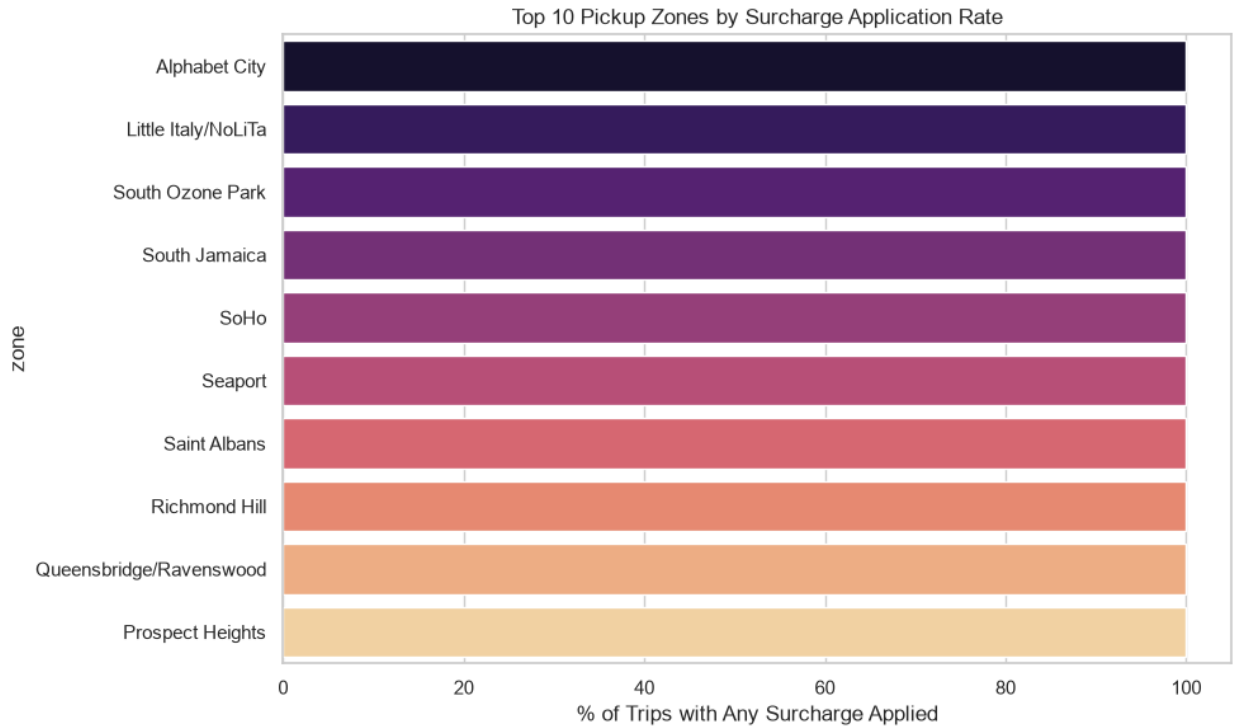
Variation by zone (cell 154): Separately for pickup and drop-off zones, the code calculated the share of trips with any surcharge. It excluded zones with fewer than 20 trips, joined the results to the taxi-zone lookup for names and boroughs, and printed the ten zones with the highest application rates. A bar chart was generated for the top pickup zones; the code prints the top drop-off zones but does not plot them.

Top 10 pickup zones by surcharge application rate:

	zone	borough	surcharge_rate	trip_count
0	Alphabet City	Manhattan	100.00	258
53	Little Italy/NoLiTa	Manhattan	100.00	2380
77	South Ozone Park	Queens	100.00	45
76	South Jamaica	Queens	100.00	48
75	SoHo	Manhattan	100.00	2002
74	Seaport	Manhattan	100.00	473
73	Saint Albans	Queens	100.00	28
72	Richmond Hill	Queens	100.00	27
71	Queensbridge/Ravenswood	Queens	100.00	57
70	Prospect Heights	Brooklyn	100.00	21

Top 10 dropoff zones by surcharge application rate:

	zone	borough	surcharge_rate	trip_count
0	Newark Airport	EWB	100.00	741
159	Roosevelt Island	Manhattan	100.00	157
135	North Corona	Queens	100.00	57
136	Norwood	Bronx	100.00	26
137	Oakland Gardens	Queens	100.00	29
138	Ocean Hill	Brooklyn	100.00	68
139	Ocean Parkway South	Brooklyn	100.00	28
140	Old Astoria	Queens	100.00	242
141	Ozone Park	Queens	100.00	29
142	Park Slope	Brooklyn	100.00	667



The analysis therefore compares surcharge application rates, not surcharge amounts or total surcharge revenue. The code's outputs are

needed to name the specific leading zones, hours, or days; those values are not included in the provided excerpt. Interpret the results with two caveats: they are based on a sample, and the code includes airport_fee only if that column exists—if airport fees were stored under a different name, they would not be included in these comparisons.

4. Conclusions

4.1. Final Insights and Recommendations

4.1.1. Recommendations to optimize routing and dispatching based on demand patterns and operational inefficiencies.

Insight	Recommendation
Rush-hour speed drops (morning & evening) on specific routes	Feed low-speed, high-frequency routes into ETA models so dispatch/navigation systems proactively add buffer time and avoid those corridors when alternatives exist
Congestion concentrated in Manhattan core during peak hours	Use dynamic routing that reroutes drivers around known slow corridors during 7–9 AM and 4–7 PM windows rather than defaulting to shortest-distance paths
Dead zone (4–5 AM) has minimal demand	Schedule vehicle maintenance, shift changes, and driver breaks during this window to avoid idle capacity during low-demand hours
Night-hour top zones differ from daytime hubs (nightlife/transit vs. business)	Build time-conditional dispatch logic that shifts driver-facing "hot zone" recommendations based on time of day, not a static heatmap
Airport and event-driven trips show higher passenger counts	Prioritize larger-capacity vehicle dispatch to zones/times with historically higher average passenger counts (nightlife, airports, weekend evenings)

4.1.2. Suggestions on strategically positioning cabs across different zones to make best use of insights uncovered by analysing trip trends across time, days and months.

Insight	Recommendation
Weekday demand peaks sharply at 6 PM; weekend demand is a flatter plateau	Concentrate maximum fleet availability in Manhattan business zones for the weekday 3–7 PM window; spread availability more evenly across a longer midday-to-evening window on weekends
Certain zones are net "originators" (high pickup/dropoff ratio) vs. net "destinations"	Use origin-heavy zones as morning staging points and destination-heavy zones as afternoon/evening staging points, effectively pre-positioning supply ahead of predictable directional flow (e.g., residential-to-business in the morning, reverse in the evening)
Night-hour top zones differ from daytime hubs	Reposition a portion of the fleet toward nightlife/transit hub zones after 11 PM rather than leaving vehicles idle in daytime business districts
Higher passenger counts cluster near airports and entertainment districts	Maintain a standing allocation of higher-capacity vehicles near these zones, especially on weekend evenings and late nights
Low-volume dead zone (4–5 AM) is citywide, not zone-specific	Use this window for empty-leg repositioning — moving vehicles from the previous night's demand zones back toward the next morning's expected origin zones (e.g., residential areas) at minimal opportunity cost

4.1.3. Propose data-driven adjustments to the pricing strategy to maximize revenue while maintaining competitive rates with other vendors.

Insight	Recommendation
Fare per mile decreases as trip distance increases (fixed-charge dilution)	Consider a modest minimum-distance surcharge or fare floor adjustment for very short trips to better cover fixed dispatch costs, without altering the competitive metered structure for longer trips
Fare per mile per passenger drops as occupancy increases	Introduce a shared-ride/pooled pricing tier or small per-passenger surcharge, capturing more revenue per vehicle-mile on higher-occupancy trips without pricing out any single rider
Night trips are low-volume but carry higher average fares/revenue-per-trip	Maintain or slightly reinforce existing night/overnight surcharge levels — they're already an efficient way to monetize a low-volume window without needing broader fare hikes
Surcharges concentrate in rush hour and Manhattan-core zones	This is already effectively a de facto dynamic pricing mechanism; formalize it further by tying surge/extra-charge intensity more tightly to real-time congestion data rather than fixed time windows, to stay competitive while still capturing peak-demand value
Fares are largely vendor-agnostic and metered	Since base pricing is regulated and not a meaningful differentiator, competitive edge should come from service-level factors (wait time, ETA accuracy, vehicle availability) rather than undercutting fares — reinforcing that the routing/positioning recommendations above directly support the pricing strategy's competitiveness
Tip percentage data is biased by cash payment underreporting at night	Exclude or caveat tip-based profitability metrics for night hours when making revenue decisions, since apparent "low-tip" periods are partly a reporting artifact, not a true demand signal

End of the Report

-Author

Jahangir Pasha

Email – Jahangir.Pasha.2006@gmail.com